UNIVERSIDAD NACIONAL DE INGENIERÍA

Programacion 2

INTEGRANTES: Alexander Espinoza Mejía

Carlos Alexander Mendez Ampie

Josep zabub Hiram subotei Andradez Pineda

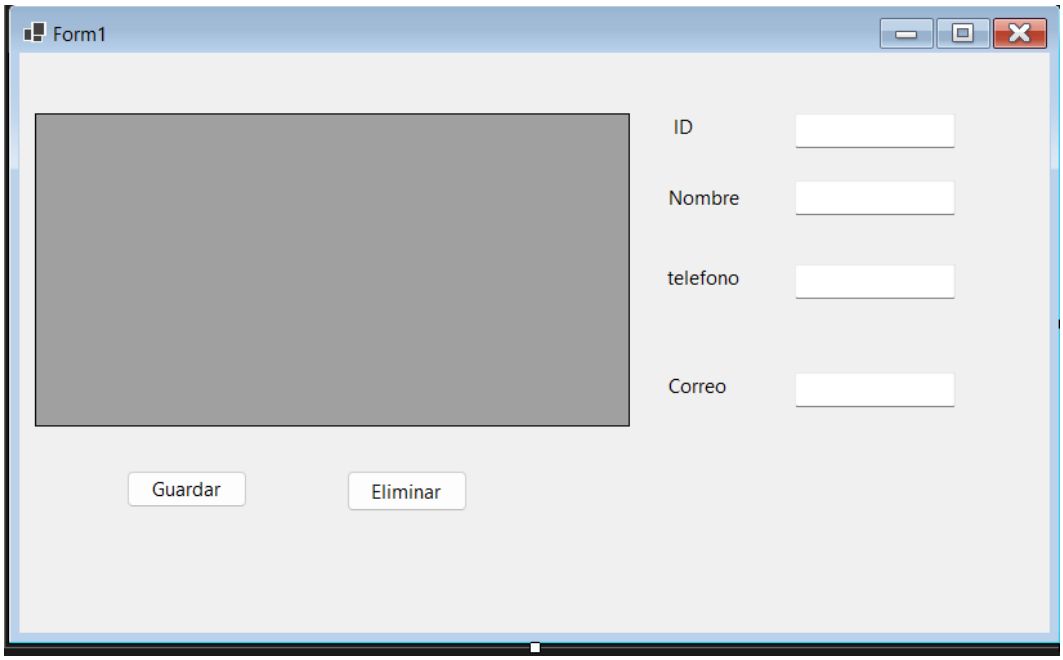
Ivan Martinez

DOCENTE: Abel Marin

FECHA: 8/5/2026

Managua, Nicaragua.

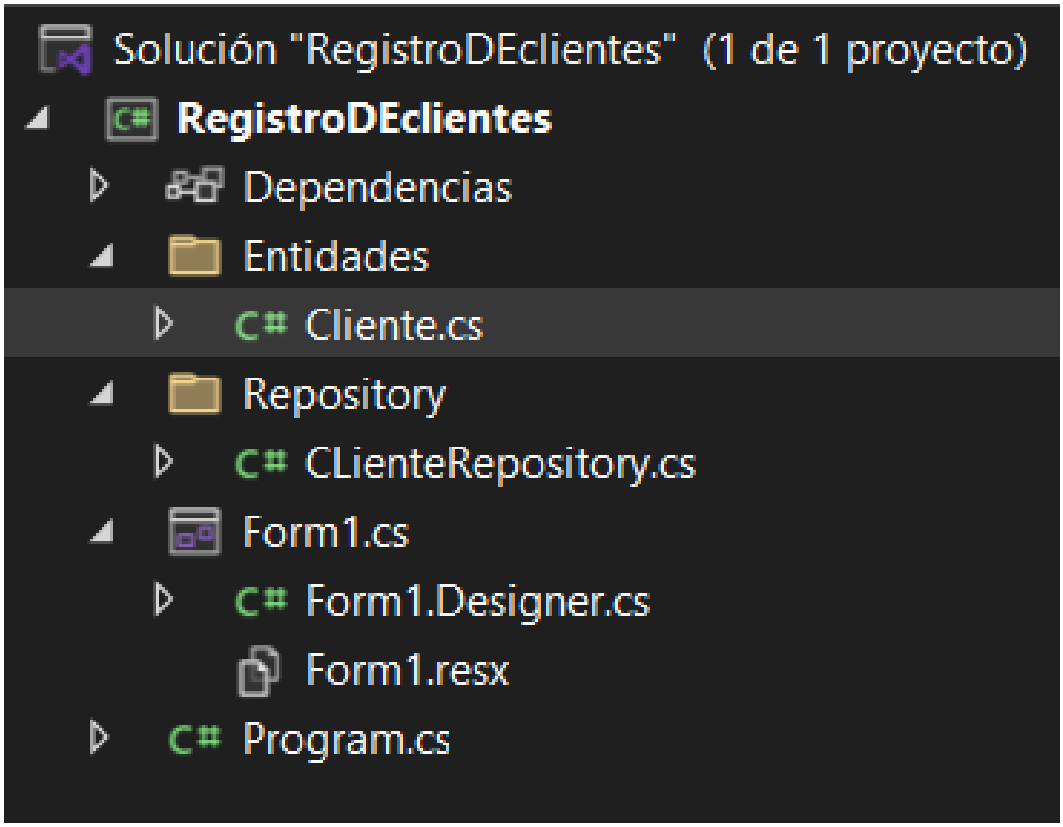
Ejercicio 1



En la primera imagen se muestra el diseño principal del formulario en Windows Forms. A la izquierda se colocó un DataGridView, que sirve para mostrar en forma de tabla los clientes registrados en la base de datos. A la derecha se agregaron campos de entrada para ID, Nombre, teléfono y Correo, donde el usuario puede escribir o consultar la información. En la parte inferior están los botones Guardar y Eliminar, que se usarán para registrar nuevos datos o quitar un cliente seleccionado.

```
namespace RegistroDEclientes.Entidades
{
    3 referencias
    public  class Cliente
    {

        2 referencias
        public  int Id { get; set; }
        2 referencias
        public string Nombre { get; set; }
        2 referencias
        public string  Telefono { get; set; }

        2 referencias
        public string Correo {  get; set; }
```

La clase Cliente, ubicada dentro del espacio de nombres del proyecto. Esta clase funciona como el modelo de datos de la aplicación, porque define las propiedades que tendrá cada cliente: Id, Nombre, Teléfono y Correo. Su función es organizar la información antes de enviarla a la base de datos o mostrarla en el formulario, evitando manejar los datos de forma desordenada dentro del código.



Aquí se muestra el Explorador de soluciones de Visual Studio con la estructura del proyecto. Se observa una carpeta llamada Entidades, donde está la clase Cliente, y una carpeta Repository, donde se encuentra la clase ClienteRepository. Esta organización separa la información del cliente, la conexión con la base de datos y el formulario principal. Sirve para que el proyecto sea más ordenado y más fácil de mantener.

```
public class CLienteRepository
{

    private readonly string conexion;

    3 referencias
    public CLienteRepository(string conexion)
    {


        this.conexion = conexion;

    }
```

En esta imagen aparece el inicio de la clase ClienteRepository. Se declara una variable privada llamada conexion, que guardará la cadena de conexión hacia SQL Server. También se crea el constructor de la clase, el cual recibe esa conexión y la almacena para que pueda ser usada en los métodos del repositorio. Esto permite que todas las operaciones de guardar, consultar o eliminar datos trabajen con la misma conexión.

```
public void Guardar(Cliente cliente)
{
    using (SqlConnection conn = new SqlConnection(conexion))
    {
        conn.Open();

        string query = "INSERT INTO Clientes (Id,Nombre, Telefono, Correo) VALUES (@Id,@Nombre, @Telefono, @Correo)";

        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@Id", cliente.Id);
        cmd.Parameters.AddWithValue("@Nombre", cliente.Nombre);
        cmd.Parameters.AddWithValue("@Telefono", cliente.Telefono);
        cmd.Parameters.AddWithValue("@Correo", cliente.Correo);

        cmd.ExecuteNonQuery();
    }
}
```

En esta parte se muestra el método Guardar, encargado de insertar un nuevo cliente en la tabla Clientes. Primero se abre la conexión con SQL Server y luego se prepara una consulta INSERT INTO con los campos Id, Nombre, Teléfono y Correo. Se usan parámetros como @Id, @Nombre, @Telefono y @Correo para

enviar los valores de forma más segura y ordenada. Finalmente, ExecuteNonQuery ejecuta la instrucción y guarda el registro en la base de datos.

```
public DataTable ObtenerTodos()
{
    using (SqlConnection conn = new SqlConnection(conexion))
    {
        conn.Open();

        SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Clientes", conn);
        DataTable dt = new DataTable();
        da.Fill(dt);

        return dt;
    }
}
```

En esta imagen se muestra el método ObtenerTodos, que se encarga de consultar todos los registros guardados en la tabla Clientes. El código abre la conexión, ejecuta una consulta SELECT * FROM Clientes y usa un SqlDataAdapter para llenar un DataTable. Ese DataTable se devuelve al formulario para que la información pueda mostrarse dentro del DataGridView. Este método sirve para cargar y actualizar la lista de clientes en pantalla.

```
public void Eliminar(int id)
{
    using (SqlConnection conn = new SqlConnection(conexion))
    {
        conn.Open();

        string query = "DELETE FROM Clientes WHERE Id = @Id";

        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@Id", id);

        cmd.ExecuteNonQuery();
    }
}
```

Aquí se observa el método Eliminar, que permite borrar un cliente de la base de datos usando su Id. Primero se abre la conexión y luego se prepara una consulta DELETE FROM Clientes WHERE Id = @Id. El parámetro @Id indica exactamente

qué registro se debe eliminar, evitando borrar información equivocada. Al final, ExecuteNonQuery ejecuta la acción y elimina el cliente seleccionado.

```
public partial class Form1 : Form
{
    string conexion = "server=localhost\\SQLEXPRESS01;Database=UnidadIII;Trusted_Connection=True;";

    3 referencias
    private void CargarDatos()
    {
        CLienteRepository repo = new CLienteRepository(conexion);
        dataGridView1.DataSource = repo.ObtenerTodos();
    }
```

Tenemos una parte del código del formulario principal Form1. Primero declaramos la cadena de conexión hacia la base de datos UnidadIII en SQL Server. Luego se crea el método CargarDatos, que instancia ClienteRepository y asigna al DataGridView el resultado de ObtenerTodos. Esto permite que la tabla del formulario muestre automáticamente los clientes guardados cada vez que se cargan o actualizan los datos.

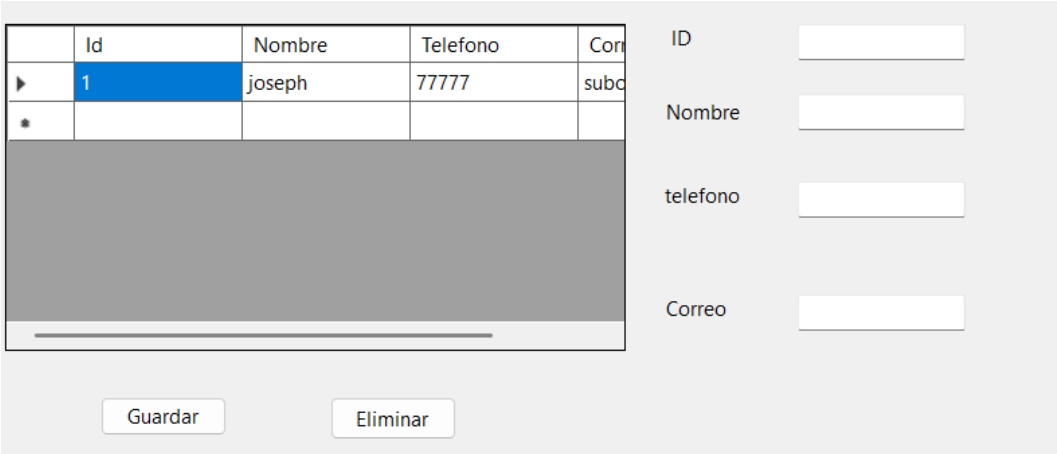
```
1   create database UnidadIII;
2   go
3
4   use UnidadIII;
5   go
6
7   create table CLientes(
8       Id int primary key,
9       Nombre nvarchar(100) not null,
10      Telefono int not null,
11      Correo nvarchar(100) not null
12  );
```

En esta imagen se muestra el script SQL utilizado para crear la base de datos y la tabla donde se almacenarán los clientes. Primero se crea la base de datos

UnidadIII y luego se selecciona para trabajar sobre ella. Después se crea la tabla Clientes con los campos Id, Nombre, Teléfono y Correo. El campo Id se define como llave primaria, lo que significa que identifica de forma única a cada cliente dentro de la tabla.

```
private void btnEliminar_Click(object sender, EventArgs e)
{

    if (dataGridView1.CurrentRow != null)
    {
        int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Id"].Value);

        CLienteRepository repo = new CLienteRepository(conexion);
        repo.Eliminar(id);

        MessageBox.Show("Eliminado");

        CargarDatos();
    }
}
```

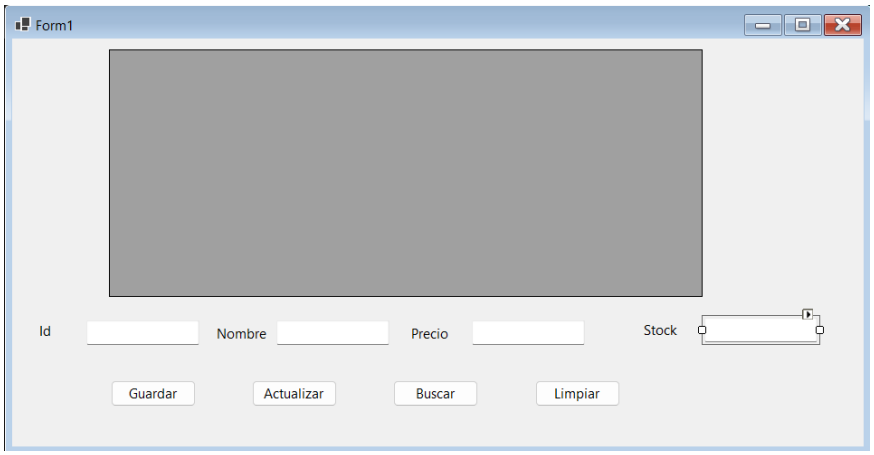
En esta parte aparece el código del botón Eliminar dentro del formulario. Antes de borrar, el programa verifica que exista una fila seleccionada en el DataGridView. Luego obtiene el Id del cliente seleccionado, crea una instancia de ClienteRepository y llama al método Eliminar. Después muestra un mensaje indicando que el cliente fue eliminado y vuelve a cargar los datos para actualizar la tabla en pantalla.



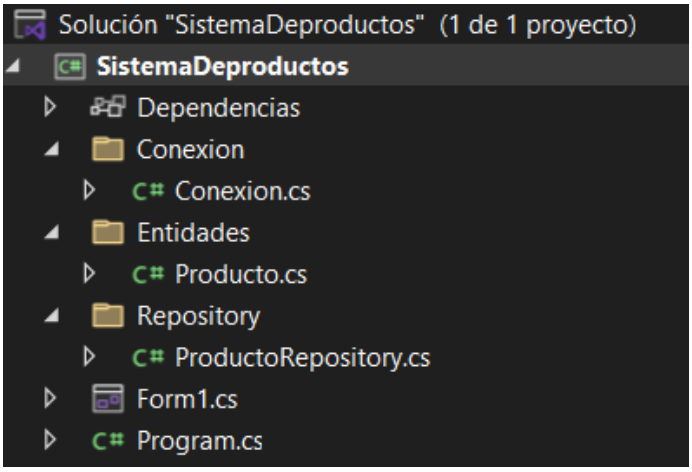
Ya ejecutándose se tiene que ver así si no tenemos errores. El DataGridView muestra un registro de cliente guardado, con su Id, nombre, teléfono y correo. A la derecha permanecen los campos de entrada y abajo los botones Guardar y Eliminar. Esta captura demuestra que la conexión entre el formulario, el código y la base de datos funciona correctamente, ya que la información se muestra en la interfaz del programa.

Ejercicio 2

Este proyecto fue desarrollado utilizando Windows Forms como interfaz gráfica y Entity Framework como tecnología de acceso a datos.



Primero hacemos muestra del diseño del formulario principal en Windows Forms. Contiene un DataGridView para visualizar los datos y campos de entrada (Id, Nombre, Precio, Stock), junto con botones para ejecutar operaciones CRUD.



Estructura del proyecto en Visual Studio. Creamos 3 carpetas organizadas por capas: conexión la cuál será la base de datos, esta carpeta contendrá a la clase Conexion, Entidades que contendrá la clase Producto, Repository que contendrá a la clase ProductoRepository que es la lógica de acceso a datos y también está el formulario principal.

```
public  class Producto
{
    1 referencia
    public int Id { get; set; }
    3 referencias
    public string Nombre { get; set; }
    2 referencias
    public decimal Precio { get; set; }
    2 referencias
    public int Stock { get; set;}
}
```

Definición de la entidad Producto. Esta clase representa la estructura de un producto tanto en el sistema como en la base de datos. Representa la tabla en base de datos con propiedades: Id, Nombre, Precio y Stock.

```
public class AppDbContext : DbContext
{
    4 referencias
    public DbSet<Producto> Productos { get; set; }

    0 referencias
    protected override void OnConfiguring(DbContextOptionsBuilder options)
    {
        options.UseSqlServer(
            "Server=localhost\\SQLEXPRESS01;Database=EJERCICIO2;Trusted_Connection=True;TrustServerCertificate=True;"
        );
    }
}
```

Definición del DbContext de Entity Framework la cual es la encargada de gestionar la conexión entre la aplicación y la base de datos utilizando Entity Framework.

AppDbContext Esta clase hereda de DbContext, lo que permite trabajar con la base de datos de una forma más sencilla mediante objetos en lugar de consultas SQL directas.

Aquí se configura la conexión a SQL Server y se declara el DbSet de productos.

```
public class ProductoRepository
{
    AppDbContext context = new AppDbContext();

    1 referencia
    public void Insertar(Producto p)
    {
        context.Productos.Add(p);
        context.SaveChanges();
    }
```

Método Insertar en el repositorio. Permite agregar un nuevo producto a la base de datos usando Entity Framework.

```
public void Actualizar(Producto p)
{
    context.Productos.Update(p);
    context.SaveChanges();
}
```

Método Actualizar en el repositorio. Permite modificar un producto existente.

```
public List<Producto> Listar()
{
    return context.Productos.ToList();
}
1 referencia
public List<Producto> Buscar(string nombre)
{
    return context.Productos
        .Where(x => x.Nombre.Contains(nombre))
        .ToList();
}
```

Métodos Listar y Buscar. Listar obtiene todos los registros y Buscar filtra productos por nombre.

```
public partial class Form1 : Form
{
    Repository.ProductoRepository repo = new Repository.ProductoRepository();
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }
```

En este punto se realiza la inicialización del formulario y se crea una instancia del repositorio de productos, el cual será utilizado para interactuar con la base de datos.

El repositorio permite encapsular la lógica de acceso a datos, facilitando operaciones como insertar, actualizar o consultar registros sin necesidad de manejar directamente la conexión desde el formulario.

```
private void btnGuardar_Click(object sender, EventArgs e)
{
    if (txtNombre.Text == "" || txtPrecio.Text == "" || txtStock.Text == "")
    {
        MessageBox.Show("Campos vacíos");
        return;
    }
    Producto p = new Producto
    {
        Nombre = txtNombre.Text,
        Precio = decimal.Parse(txtPrecio.Text),
        Stock = int.Parse(txtStock.Text)
    };
    repo.Insertar(p);
    MessageBox.Show("Guardado");
    dgvProductos.DataSource = repo.Listar();
}
```

Evento del botón Guardar este evento valida campos, crea un objeto Producto y lo inserta en la base de datos.

```
    private void btnActualizar_Click(object sender, EventArgs e)
    {
        if(txtId.Text == "" || txtNombre.Text == "" || txtPrecio.Text == "" || txtStock.Text == "")
{
            MessageBox.Show("Completa todos los campos");
            return;
        }
        int id;
        decimal precio;
        int stock;
        if (!int.TryParse(txtId.Text, out id) ||
            !decimal.TryParse(txtPrecio.Text, out precio) ||
            !int.TryParse(txtStock.Text, out stock))
        {
            MessageBox.Show("Datos inválidos");
            return;
        }
        Producto p = new Producto
        {
            Id = id,
            Nombre = txtNombre.Text,
            Precio = precio,
            Stock = stock
        };
        repo.Actualizar(p);
        dgvProductos.DataSource = repo.Listar();
    }
```

Evento del botón Actualizar. Valida datos y actualiza el producto seleccionado. Se valida que todos los campos del formulario estén completos; en caso contrario, se muestra un mensaje al usuario.

```
private void btnBuscar_Click(object sender, EventArgs e)
{
    dgvProductos.DataSource = repo.Buscar(txtNombre.Text);
}
1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtId.Clear();
    txtNombre.Clear();
    txtPrecio.Clear();
    txtStock.Clear();
}
```

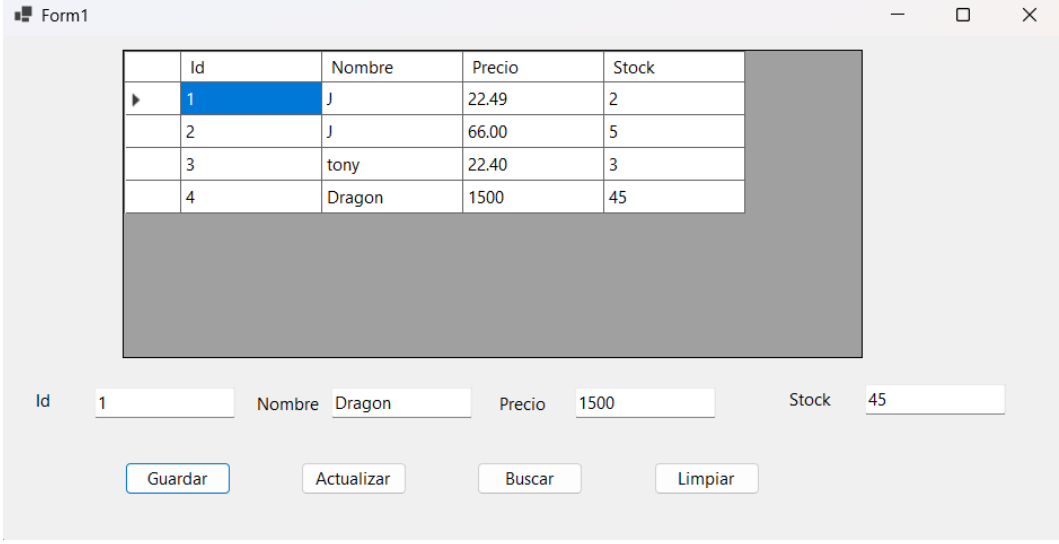
Evento Buscar y Limpiar. Permiten mostrar los datos en el DataGridView de los productos en función del nombre ingresado por el usuario, utilizando el método del repositorio y actualizando el contenido del DataGridView con los resultados obtenidos y por otro lado el botón limpiar se encarga de restablecer los campos del formulario, eliminando cualquier valor ingresado previamente.

```
private void dgvProductos_CellContentClick(object sender, DataGridViewCellEventArgs e)
{


    txtId.Text = dgvProductos.CurrentRow.Cells[0].Value.ToString();
    txtNombre.Text = dgvProductos.CurrentRow.Cells[1].Value.ToString();
    txtPrecio.Text = dgvProductos.CurrentRow.Cells[2].Value.ToString();
    txtStock.Text = dgvProductos.CurrentRow.Cells[3].Value.ToString();
}
```

Evento del DataGridView. El cual se ejecuta cuando el usuario selecciona una celda dentro de la tabla de productos. Su función es tomar los datos de la fila seleccionada y cargarlos automáticamente en los campos del formulario. Cuando el usuario hace clic sobre un registro, la información correspondiente Id, Nombre, Precio y Stock se copia en los cuadros de texto, permitiendo visualizar y editar fácilmente los datos del producto.

```sql
create database EJERCICIO2;
go

use EJERCICIO2;
go

create table Productos(
    id int primary key,
    nombre nvarchar(100) not null,
    Precio decimal(20, 2) not null,
    Stock int
);
```

Script SQL para crear la base de datos y la tabla Productos. Establece la estructura de la base de datos que será utilizada, permitiendo que Entity Framework pueda vincular la información de las clases con las tablas de SQL y realizar operaciones sobre los datos.

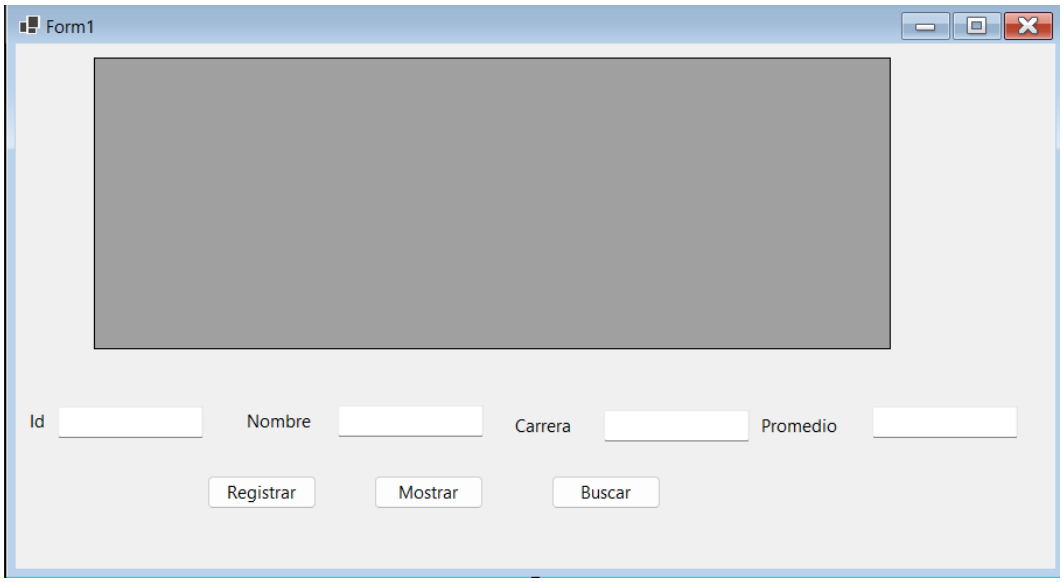


Así nos debería quedar el resultado final de la aplicación en ejecución. Se visualizan registros cargados en el DataGridView y un producto seleccionado listo para editar.

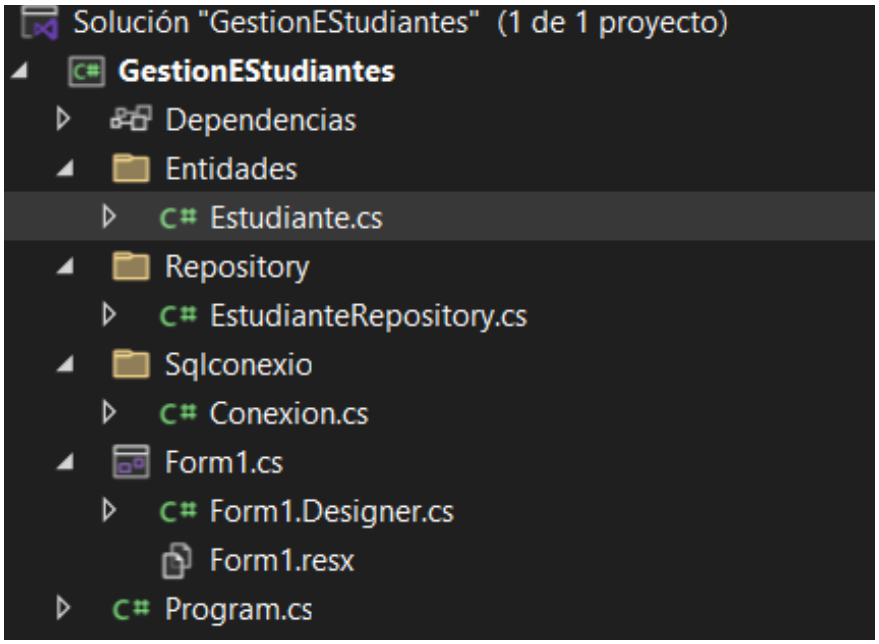
Ejercicio 3

Lo hicimos con Windows Forms como interfaz gráfica y ADO.NET como acceso a datos. A diferencia de Entity Framework, en este caso la conexión a la base de datos y la ejecución de consultas SQL se realizan manualmente mediante clases

como SqlConnection, SqlCommand y SqlDataReader.



Hacemos el diseño del formulario principal, donde se ingresan los datos del estudiante y se visualizan en una tabla, el cual es una DataGridview.



Se observa la estructura del proyecto organizada en carpetas las cuales nombraremos como entidades, repositorio, conexión y formulario.

```
12 referencias
public class Estudiante
{
    2 referencias
    public int Id { get; set; }
    4 referencias
    public string Nombre { get; set; }
    5 referencias
    public string Carrera { get; set; }
    5 referencias
    public decimal Promedio { get; set; }
}
```

La clase Estudiante define la estructura de los datos que se manejarán en la aplicación y en la base de datos. Esta clase agrupa la información principal de cada estudiante, como su identificador, nombre, carrera y promedio.

```
namespace SistemaDeproductos.Conexion
{
    2 referencias
    public class Conexion
    {
        private string cadena = "Server=localhost\\SQLEXPRESS01;Database=EJERCICIO3;Trusted_Connection=True;TrustServerCertificate=True;";

        3 referencias
        public SqlConnection ObtenerConexion()
        {
            return new SqlConnection(cadena);
        }
    }
}
```

Se muestra la clase de conexión, encargada de establecer la comunicación con SQL Server mediante una cadena de conexión.

```
public class EstudianteRepository
{
    Conexion con = new Conexion();
    1 referencia
    public void Insertar(Estudiante e)
    {
        using (var conn = con.ObtenerConexion())
        {
            conn.Open();

            string query = "INSERT INTO Estudiantes (Nombre,Carrera,Promedio) VALUES (@Nombre,@Carrera,@Promedio)";
            SqlCommand cmd = new SqlCommand(query, conn);

            cmd.Parameters.AddWithValue("@Nombre", e.Nombre);
            cmd.Parameters.AddWithValue("@Carrera", e.Carrera);
            cmd.Parameters.AddWithValue("@Promedio", e.Promedio);

            cmd.ExecuteNonQuery();
        }
    }
```

El método Insertar utiliza SqlCommand para ejecutar una consulta SQL que registra un nuevo estudiante en la base de datos. Se utilizan parámetros para enviar los datos del objeto Estudiante hacia la consulta, lo que permite realizar la inserción de forma segura y organizada. Finalmente, el comando se ejecuta para guardar la información en la tabla correspondiente.

```
2 referencias
public List<Estudiante> ListarOrdenados()
{
    List<Estudiante> lista = new List<Estudiante>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        SqlCommand cmd = new SqlCommand("SELECT * FROM Estudiantes", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Estudiante
            {
                Id = (int)dr["Id"],
                Nombre = dr["Nombre"].ToString(),
                Carrera = dr["Carrera"].ToString(),
                Promedio = (decimal)dr["Promedio"]
            });
        }
    }
    return lista.OrderByDescending(x => x.Promedio).ToList();
}
```

Se implementa un método el método ListarOrdenados, el cual se encarga de obtener todos los registros de la tabla *Estudiantes* desde la base de datos. Pero se necesita establecer una conexión y se ejecuta una consulta SQL de tipo *SELECT*, utilizando un objeto y se agrega una lista que almacena todos los resultados. Finalmente, la lista es ordenada de forma descendente según el promedio antes de ser retornada.

```
public List<Estudiante> BuscarPorCarrera(string carrera)
{
    return ListarOrdenados()
        .Where(x => x.Carrera.ToLower().Contains(carrera.ToLower()))
        .ToList();
}

1 referencia
public List<Estudiante> Listar()
{
    List<Estudiante> lista = new List<Estudiante>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        SqlCommand cmd = new SqlCommand("SELECT * FROM Estudiantes", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Estudiante
            {
                Id = (int)dr["Id"],
                Nombre = dr["Nombre"].ToString(),
                Carrera = dr["Carrera"].ToString(),
                Promedio = (decimal)dr["Promedio"]
            });
        }
    }
    return lista;
}
```

Aquí se muestra la lógica de filtrado y listado de estudiantes, incluyendo ordenamiento y búsqueda por carrera.

Los métodos BuscarPorCarrera y Listar, los cuales permiten consultar y filtrar los datos de los estudiantes desde la base de datos. El método Listar se encarga de ejecutar una consulta SQL para obtener todos los registros, utilizando un SqlDataReader para leer los datos

```
public partial class Form1 : Form
{
    EstudianteRepository repository = new EstudianteRepository();
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }
```

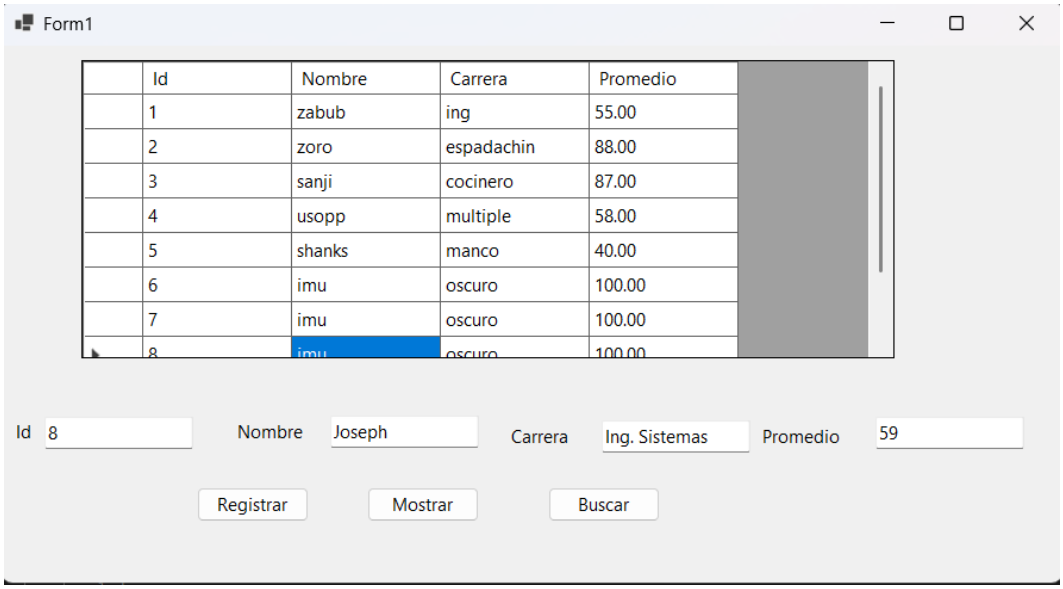
Se inicializa el formulario principal y se crea la instancia del repositorio para acceder a los datos.

```
private void btnRegistrar_Click(object sender, EventArgs e)
{
    if (txtNombre.Text == "" || txtCarrera.Text == "" || txtPromedio.Text == "")
    {
        MessageBox.Show("Campos vacíos");
        return;
    }
    decimal promedio;
    if (!decimal.TryParse(txtPromedio.Text, out promedio))
    {
        MessageBox.Show("Promedio inválido");
        return;
    }
    Estudiante e1 = new Estudiante
    {
        Nombre = txtNombre.Text,
        Carrera = txtCarrera.Text,
        Promedio = promedio
    };
    repository.Insertar(e1);

    MessageBox.Show("Registrado");
    dgvEstudiantes.DataSource = repository.Listar();
}
```

El evento del botón Registrar, el cual se encarga de capturar y validar los datos ingresados por el usuario en el formulario, se verifica que los campos no estén vacíos; en caso que así sea se muestra un mensaje de advertencia. También se valida que el valor del promedio sea numérico utilizando una conversión. Si los datos son válidos, se crea un objeto Estudiante con la información ingresada y se envía al repositorio para después aparezca en la base de datos y se muestra un mensaje de confirmación, se actualiza el contenido del DataGridView para reflejar el nuevo registro.

```
1 referencia
private void btnMostrar_Click(object sender, EventArgs e)
{
    dgvEstudiantes.DataSource = repository.ListarOrdenados();
}

1 referencia
private void btnBuscar_Click(object sender, EventArgs e)
{
    dgvEstudiantes.DataSource = repository.BuscarPorCarrera(txtCarrera.Text);
}
```

Los botones Mostrar y Buscar permiten listar los estudiantes y filtrar los resultados según la carrera.

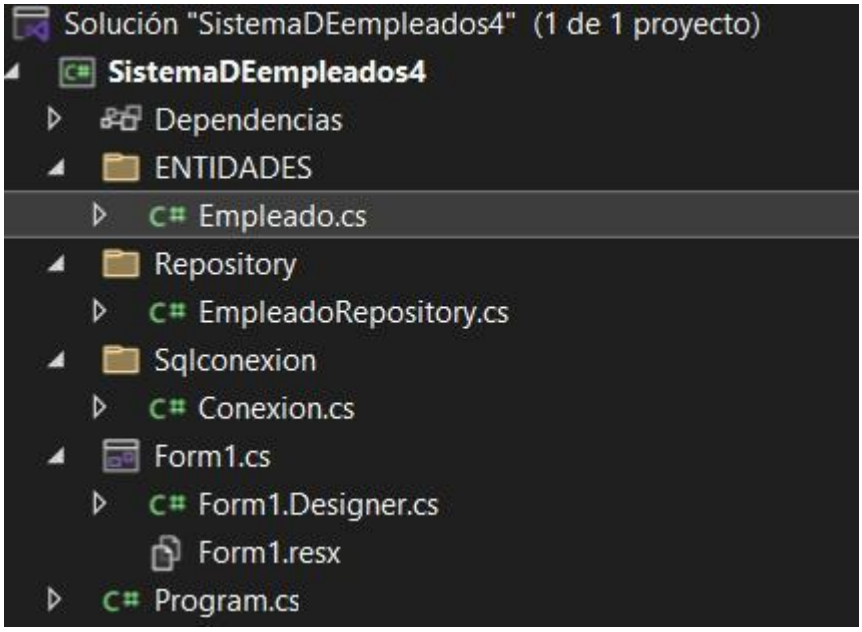


Al final si no tenemos errores de conexión así nos debería aparecer, se muestra la ejecución final de la aplicación con los datos cargados en el DataGridView.

Ejercicio 4

Primero creamos nuestra interfaz gráfica

Ahora creamos tres carpetas, Entidades que contiene una clase llamada Empleados.cs, Repository que contiene una clase llamada EmpleadoRepository, Sqlconexion que contiene una clase llamada Conexión.cs



Ahora nos dirigimos a la clase Empleado donde vamos a declarar nuestros atributos

```
namespace SistemaDEempleados4.ENTIDADES
{
    11 referencias
    public  class Empleado
    {
        3 referencias
        public int Id { get; set; }
        6 referencias
        public string Nombre { get; set; }
        5 referencias
        public string Cargo { get; set; }
        8 referencias
        public decimal Salario { get; set; }
    }
}
```

Ahora vamos a la clase conexión, esta encarga de establecer la conexión con la base de datos SQL Server mediante una cadena de conexión. El método ObtenerConexion() crea y devuelve un objeto SqlConnection

```
namespace SistemaDEempleados4.Sqlconexion
{
    2 referencias
    public class Conexion
    {
        private string cadena = "Server=localhost\\SQLEXPRESS01;Database=EmpresaDB;Trusted_Connection=True;TrustServerCertificate=True;";
        4 referencias
        public SqlConnection ObtenerConexion()
        {
            return new SqlConnection(cadena);
        }
    }

}
```

Ahora nos vamos a la clase EmpleadoRepository, el método agrrgar se encarga de insertar un nuevo empleado en la base de datos, también abre una conexión y ejecuta una consulta SQL usando parámetros y guarda los datos del objeto

```
public class EmpleadoRepository
{
    private Conexion con = new Conexion();

    1 referencia
    public void Agregar(Empleado emp)
    {
        using (var conn = con.ObtenerConexion())
        {
            conn.Open();

            string query = "INSERT INTO Empleados (Nombre, Cargo, Salario) VALUES (@Nombre, @Cargo, @Salario)";
            SqlCommand cmd = new SqlCommand(query, conn);

            cmd.Parameters.AddWithValue("@Nombre", emp.Nombre);
            cmd.Parameters.AddWithValue("@Cargo", emp.Cargo);
            cmd.Parameters.AddWithValue("@Salario", emp.Salario);

            cmd.ExecuteNonQuery();
        }
    }
```

Empleado.

El método Listar se encarga de obtener todos los empleados de la base de datos. Ejecuta una consulta SELECT, recorre los resultados y los convierte en una lista de objetos Empleado para usarlos en el programa.

```
public List<Empleado> Listar()
{
    List<Empleado> lista = new List<Empleado>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        SqlCommand cmd = new SqlCommand("SELECT * FROM Empleados", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Empleado
            {
                Id = (int)dr["Id"],
                Nombre = dr["Nombre"].ToString(),
                Cargo = dr["Cargo"].ToString(),
                Salario = (decimal)dr["Salario"]
            });
        }
    }

    return lista;
}
```

El método totalsalario calcula la suma total de los salarios de todos los empleados. Utiliza la lista obtenida del método Listar y suma el campo Salario de cada uno.

El metido mayorsalario obtiene el empleado con el salario más alto. Ordena la lista de empleados de mayor a menor según su salario y devuelve el primero.

```
public decimal TotalSalarios()
{
    return Listar().Sum(e => e.Salario);
}
1 referencia
public Empleado MayorSalario()
{
    return Listar().OrderByDescending(e => e.Salario).FirstOrDefault();
}
```

El método actualizar se encarga de modificar los datos de un empleado en la base de datos. Ejecuta una consulta UPDATE usando parámetros para actualizar nombre, cargo y salario según el Id del empleado.

```
public void Actualizar(Empleado emp)
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        string query = "UPDATE Empleados SET Nombre=@Nombre, Cargo=@Cargo, Salario=@Salario WHERE Id=@Id";
        SqlCommand cmd = new SqlCommand(query, conn);

        cmd.Parameters.AddWithValue("@Id", emp.Id);
        cmd.Parameters.AddWithValue("@Nombre", emp.Nombre);
        cmd.Parameters.AddWithValue("@Cargo", emp.Cargo);
        cmd.Parameters.AddWithValue("@Salario", emp.Salario);

        cmd.ExecuteNonQuery();
    }
}
```

El método eliminar se encarga de eliminar un empleado de la base de datos según su Id. Ejecuta una consulta DELETE usando parámetros para evitar errores y eliminar el registro correspondiente.

```
public void Eliminar(int id)
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        SqlCommand cmd = new SqlCommand("DELETE FROM Empleados WHERE Id=@Id", conn);
        cmd.Parameters.AddWithValue("@Id", id);

        cmd.ExecuteNonQuery();
    }
}
```

Ahora vamos a nuestro Form1.cs donde se crea una instancia de EmpleadoRepository para acceder a los datos. El método CargarDatos obtiene

```
public partial class Form1 : Form
{
    EmpleadoRepository repo = new EmpleadoRepository();
    4 referencias
    private void CargarDatos()
    {
        dataGridView1.DataSource = null;
        dataGridView1.DataSource = repo.Listar();
    }
    1 referencia
    public Form1()
    {
        InitializeComponent();
        CargarDatos();
    }
```

la lista de empleados y la muestra en el DataGridView, y se ejecuta al iniciar el formulario para cargar la información automáticamente.

El evento btn.Guardar se encarga de crear un nuevo objeto Empleado con los datos ingresados en los TextBox. Luego llama al método Agregar para guardarlo en la base de datos y actualiza la tabla con CargarDatos

```
1 referencia
private void btnGuardar_Click(object sender, EventArgs e)
{
    Empleado emp = new Empleado
    {
        Nombre = txtNombre.Text,
        Cargo = txtCargo.Text,
        Salario = decimal.Parse(txtSalario.Text)
    };

    repo.Agregar(emp);
    CargarDatos();
}
```

Y el btnActualizar se encarga de actualizar un empleado existente usando los datos ingresados. Llama al método Actualizar del repositorio y recarga los datos para reflejar los cambios.

```
private void btnActualizar_Click(object sender, EventArgs e)
{
    Empleado emp = new Empleado
    {
        Id = int.Parse(txtId.Text),
        Nombre = txtNombre.Text,
        Cargo = txtCargo.Text,
        Salario = decimal.Parse(txtSalario.Text)
    };

    repo.Actualizar(emp);
    CargarDatos();
}
```

el btnEliminar se encarga de eliminar un empleado según el Id. Llama al método Eliminar del repositorio y luego actualiza la tabla con CargarDatos.

```
1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    repo.Eliminar(int.Parse(txtId.Text));
    CargarDatos();
}
```

El btnTotalSalario muestra en pantalla la suma total de los salarios de todos los empleados. Utiliza el método TotalSalarios del repositorio y lo presenta en un MessageBox.

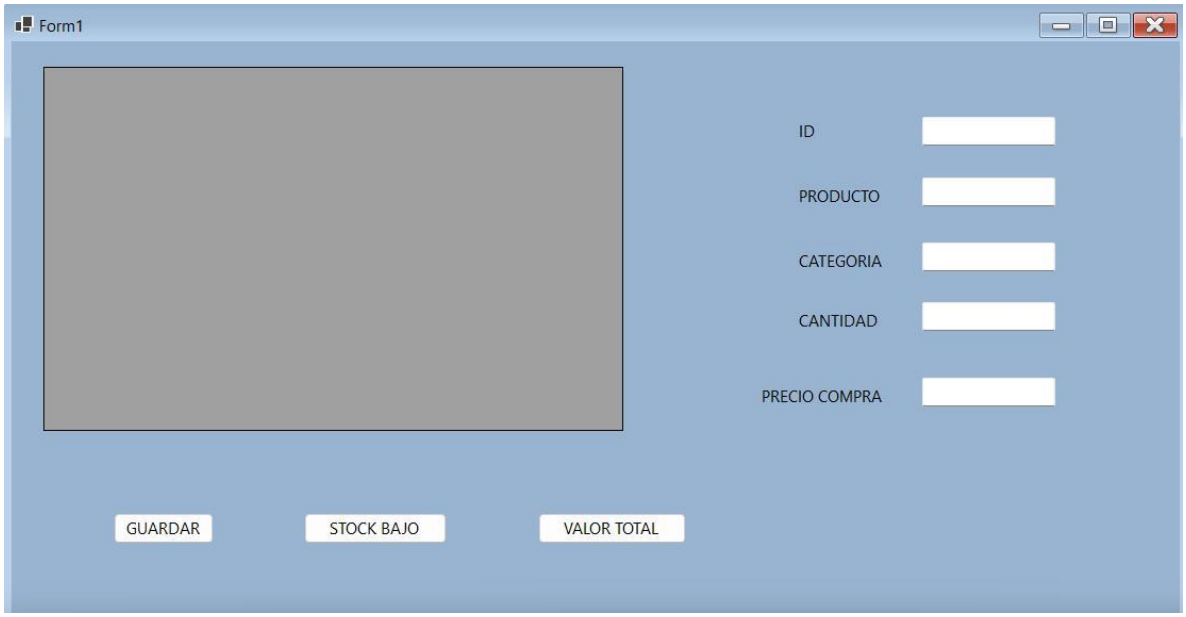
Y el btnMayorSalario obtiene el empleado con el salario más alto y muestra su nombre y salario en un mensaje. Usa el método MayorSalario y valida que exista un resultado antes de mostrarlo.

```
private void btnTotalSalarios_Click(object sender, EventArgs e)
{
    MessageBox.Show("Total pagado: " + repo.TotalSalarios());
}

1 referencia
private void btnMayorSalario_Click(object sender, EventArgs e)
{
    var emp = repo.MayorSalario();

    if (emp != null)
        MessageBox.Show($"Mayor salario: {emp.Nombre} - {emp.Salario}");

}
```

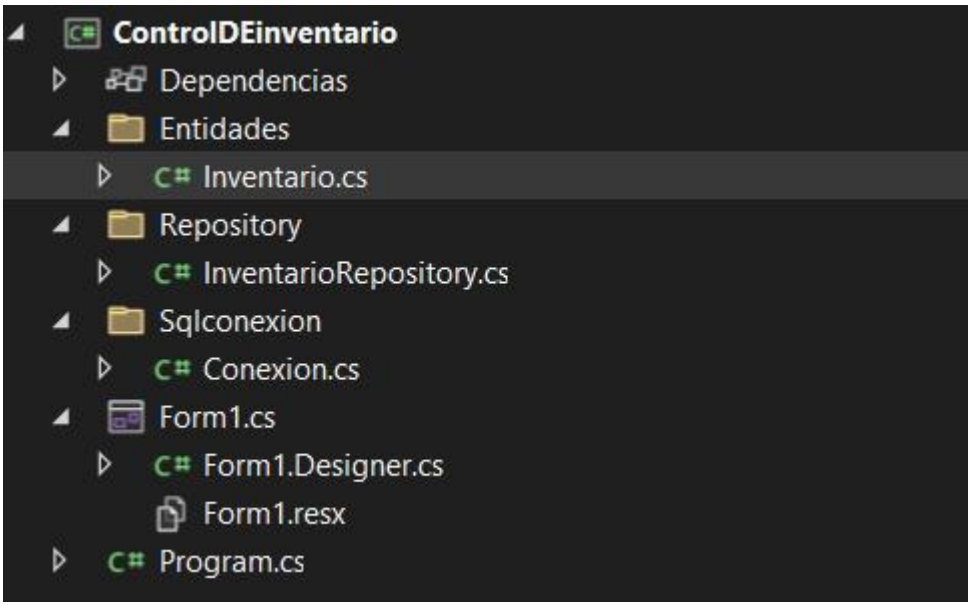
Esta sería la ejecución del código

Ejercicio 5

Primero creamos nuestra interfaz gráfica



Ahora creamos tres carpetas, Entidades que contiene una clase llamada Inventario.cs, Repository que contiene una clase llamada InventarioRepository, Sqlconexion que contiene una clase llamada Conexión.cs



Ahora nos dirigimos a la clase Empleado donde vamos a declarar nuestros atributos

```
public class Inventario
{
    1 referencia
    public int Id { get; set; }
    3 referencias
    public string Producto { get; set; }
    3 referencias
    public string Categoria { get; set; }
    5 referencias
    public int Cantidad { get; set; }
    4 referencias
    public decimal PrecioCompra { get; set; }
}
```

Ahora vamos a la clase conexión, esta encarga de establecer la conexión con la base de datos SQL Server mediante una cadena de conexión. El método ObtenerConexion() crea y devuelve un objeto SqlConnection

```
public class Conexion
{
    private string cadena = "Server=localhost\\SQLEXPRESS01;Database=InventarioDB;Trusted_Connection=True;TrustServerCertificate=True;";

    2 referencias
    public SqlConnection ObtenerConexion()
    {
        return new SqlConnection(cadena);
    }
}
```

Ahora vamos a InventarioRepository donde el método agregar se encarga de insertar un nuevo producto en la base de datos. Abre la conexión, ejecuta una consulta INSERT con parámetros y guarda los datos del objeto Inventario.

```
public  class InventarioRepository
{
    private Conexion con = new Conexion();
    1 referencia
    public void Agregar(Inventario inv)
    {
        using (var conn = con.ObtenerConexion())
        {
            conn.Open();

            string query = "INSERT INTO Inventario (Producto, Categoria, Cantidad, PrecioCompra) VALUES (@Producto, @Categoria, @Cantidad, @Precio)";
            SqlCommand cmd = new SqlCommand(query, conn);

            cmd.Parameters.AddWithValue("@Producto", inv.Producto);
            cmd.Parameters.AddWithValue("@Categoria", inv.Categoria);
            cmd.Parameters.AddWithValue("@Cantidad", inv.Cantidad);
            cmd.Parameters.AddWithValue("@Precio", inv.PrecioCompra);

            cmd.ExecuteNonQuery();
        }
    }
```

En el mismo InventarioRepository Creamos el método Listar el cual se encarga de obtener todos los productos de la base de datos. Ejecuta una consulta SELECT, recorre los resultados y los convierte en una lista de objetos Inventa para usarlos en el programa.

```
public List<Inventario> Listar()
{
    List<Inventario> lista = new List<Inventario>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM Inventario", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Inventario
            {
                Id = (int)dr["Id"],
                Producto = dr["Producto"].ToString(),
                Categoria = dr["Categoria"].ToString(),
                Cantidad = (int)dr["Cantidad"],
                PrecioCompra = (decimal)dr["PrecioCompra"]
            });
        }
    }
    return lista;
}
```

El método StockBajo obtiene los productos con poca cantidad en inventario. Filtra la lista y devuelve aquellos cuya cantidad es menor a 5.

Y el método CalcularValorTotal calcula el valor total del inventario. Multiplica la cantidad por el precio de cada producto y suma todos los resultados.

```
1 referencia
public List<Inventario> StockBajo()
{
    return Listar().Where(p => p.Cantidad < 5).ToList();
}
1 referencia
public decimal ValorTotal()
{
    return Listar().Sum(p => p.Cantidad * p.PrecioCompra);
}
```

Ahora nos dirigimos al Form1.cs donde se crea una instancia de InventarioRepository para manejar los datos del inventario. En el constructor del formulario se llama al método CargarTabla para mostrar los registros al iniciar.

```
public partial class Form1 : Form
{
    InventarioRepository repo = new InventarioRepository();
    1 referencia
    public Form1()
    {
        InitializeComponent();
        CargarTabla();
    }
```

Ahora creamos el botón para guardar el cual se encarga de validar los datos ingresados y crear un objeto Inventario. Si los datos son correctos, guarda el producto en la base de datos usando el repositorio y actualiza la tabla.

```
private void btnGuardar_Click(object sender, EventArgs e)
{
    int cantidad;
    decimal precio;
    if (string.IsNullOrWhiteSpace(txtProducto.Text) ||
        string.IsNullOrWhiteSpace(txtCategoria.Text) ||
        !int.TryParse(txtCantidad.Text, out cantidad) ||
        !decimal.TryParse(txtPrecio.Text, out precio))
    {
        MessageBox.Show("Datos inválidos. Verifica los campos.");
        return;
    }
    Inventario inv = new Inventario
    {
        Producto = txtProducto.Text,
        Categoria = txtCategoria.Text,
        Cantidad = cantidad,
        PrecioCompra = precio
    };
    repo.Agregar(inv);
    CargarTabla();
}
```

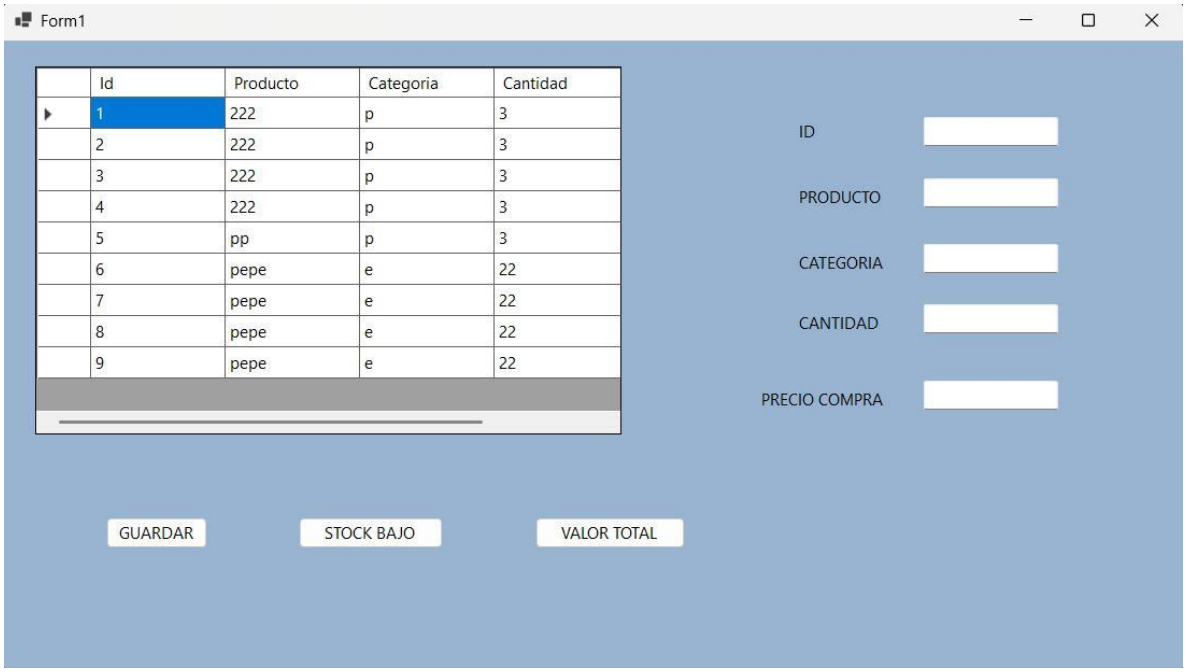
El método CargarTabla carga y muestra todos los productos del inventario en el DataGridView utilizando el método Listar.

Creamos el evento del botón de Stock bajo el cual muestra en la tabla solo los productos con bajo stock. Utiliza el método StockBajo del repositorio para filtrar los datos.

También creamos el evento del botón Valor total el cual calcula y muestra el valor total del inventario. Usa el método ValorTotal y lo presenta en un MessageBox.

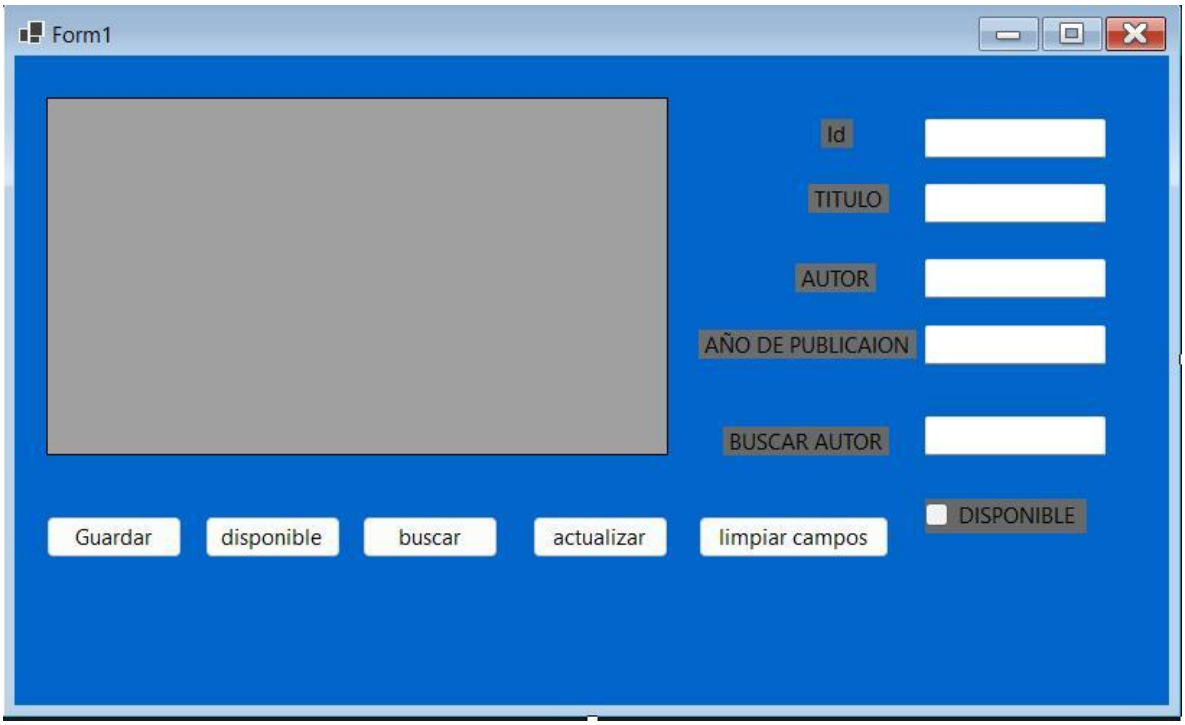
```
2 referencias
private void CargarTabla()
{
    dataGridView1.DataSource = null;
    dataGridView1.DataSource = repo.Listar();
}

1 referencia
private void btnStockBajo_Click(object sender, EventArgs e)
{
    dataGridView1.DataSource = repo.StockBajo();
}

1 referencia
private void btnValorTotal_Click(object sender, EventArgs e)
{
    MessageBox.Show("Valor total: " + repo.ValorTotal());
}
```

Y esta sería la ejecución de nuestro código

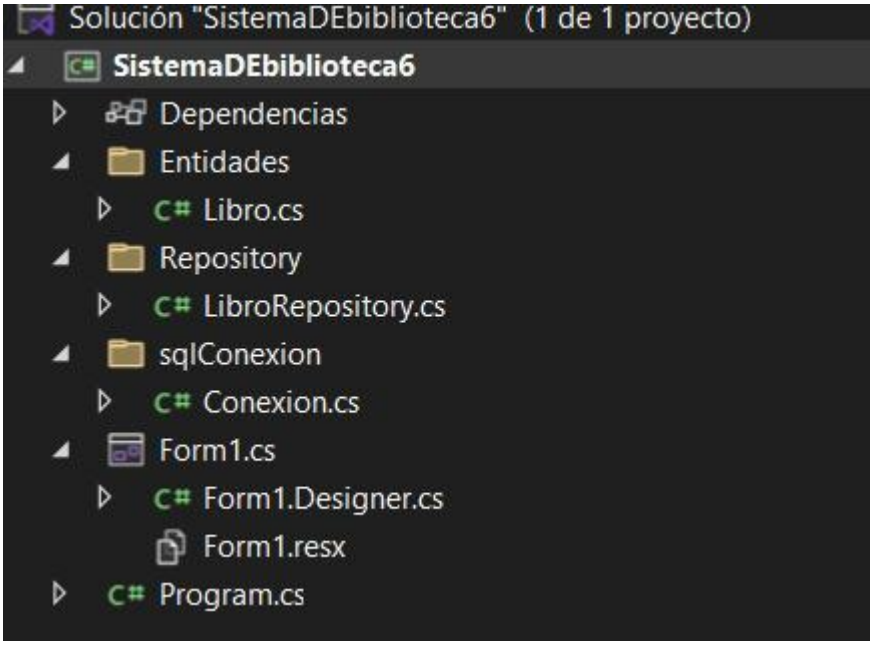


Ejercicio 6

Primero creamos nuestra interfaz gráfica



Ahora creamos tres carpetas, Entidades que contiene una clase llamada Libro.cs, Repository que contiene una clase llamada LibroRepository.cs, Sqlconexion que contiene una clase llamada Conexión.cs



Ahora nos dirigimos a la clase Libro donde vamos a declarar nuestros atributos

```
public class Libro
{
        3 referencias
        public int Id { get; set; }
        5 referencias
        public string Titulo { get; set; }
        5 referencias
        public string Autor { get; set; }
        5 referencias
        public int AñoPublicacion { get; set; }
        5 referencias
        public bool Disponible { get; set; }
}
```

Ahora vamos a la clase conexión, esta encarga de establecer la conexión con la base de datos SQL Server mediante una cadena de conexión. El método ObtenerConexion() crea y devuelve un objeto SqlConnection

```
public class Conexion
{
    private string cadena = "Server=localhost\\SQLEXPRESS01;Database=BibliotecaDB;Trusted_Connection=True;TrustServerCertificate=True;";
    5 referencias
    public SqlConnection ObtenerConexion()
    {
        return new SqlConnection(cadena);
    }
}
```

Ahora nos vamos a la clase LibroRepository, el método guardarlibro se encarga de insertar un nuevo libro en la base de datos, también abre una conexión y ejecuta una consulta SQL usando parámetros y guarda los datos del objeto Empleado.

```
public class LibroRepository
{
    private Conexion con = new Conexion();
    1 referencia
    public void Guardar(Libro libro)
    {
        using (var conn = con.ObtenerConexion())
        {
            conn.Open();
            string query = "INSERT INTO Libros (Titulo, Autor, AñoPublicacion, Disponible) VALUES (@Titulo, @Autor, @Año, @Disponible)";

            SqlCommand cmd = new SqlCommand(query, conn);
            cmd.Parameters.AddWithValue("@Titulo", libro.Titulo);
            cmd.Parameters.AddWithValue("@Autor", libro.Autor);
            cmd.Parameters.AddWithValue("@Año", libro.AñoPublicacion);
            cmd.Parameters.AddWithValue("@Disponible", libro.Disponible);

            cmd.ExecuteNonQuery();
        }
    }
```

El método ListarTodos obtiene todos los libros de la base de datos. Ejecuta una consulta SELECT, recorre los resultados y los convierte en una lista de objetos Libro.

```
1 referencia
public List<Libro> ListarTodos()
{
    List<Libro> lista = new List<Libro>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM Libros", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Libro
            {
                Id = (int)dr["Id"],
                Titulo = dr["Titulo"].ToString(),
                Autor = dr["Autor"].ToString(),
                AñoPublicacion = (int)dr["AñoPublicacion"],
                Disponible = (bool)dr["Disponible"]
            });
        }
    }
    return lista;
}
```

El método buscar autor permite buscar libros por autor. Realiza una consulta con LIKE para encontrar coincidencias y devuelve una lista de libros que cumplen con el criterio.

```
1 referencia
public List<Libro> BuscarAutor(string autor)
{
    List<Libro> lista = new List<Libro>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM Libros WHERE Autor LIKE @Autor", conn);
        cmd.Parameters.AddWithValue("@Autor", "%" + autor + "%");

        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Libro
            {
                Id = (int)dr["Id"],
                Titulo = dr["Titulo"].ToString(),
                Autor = dr["Autor"].ToString(),
                AñoPublicacion = (int)dr["AñoPublicacion"],
                Disponible = (bool)dr["Disponible"]
            });
        }
    }
    return lista;
}
```

El método actualizar disponibilidad se encarga de actualizar el estado de disponibilidad de un libro según su Id. Ejecuta una consulta UPDATE para cambiar si el libro está disponible o no.

```
1 referencia
public void ActualizarDisponibilidad(int id, bool disponible)
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("UPDATE Libros SET Disponible=@Disponible WHERE Id=@Id", conn);
        cmd.Parameters.AddWithValue("@Disponible", disponible);
        cmd.Parameters.AddWithValue("@Id", id);

        cmd.ExecuteNonQuery();
    }
}
```

El método Disponibles obtiene los libros que están disponibles en la base de datos. Ejecuta una consulta SELECT filtrando por disponibilidad y devuelve una lista de libros disponibles.

```
2 referencias
public List<Libro> Disponibles()
{
    List<Libro> lista = new List<Libro>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM Libros WHERE Disponible = 1", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Libro
            {
                Id = (int)dr["Id"],
                Titulo = dr["Titulo"].ToString(),
                Autor = dr["Autor"].ToString(),
                AñoPublicacion = (int)dr["AñoPublicacion"],
                Disponible = (bool)dr["Disponible"]
            });
        }
    }
    return lista;
}
```

El método ValidarCampos verifica que los campos obligatorios estén completos y que el año sea válido. Si hay errores, muestra un mensaje y evita continuar.

```
private bool ValidarCampos()
{
    int año;

    if (string.IsNullOrWhiteSpace(txtTitulo.Text) ||
        string.IsNullOrWhiteSpace(txtAutor.Text) ||
        !int.TryParse(txtAño.Text, out año))
    {
        MessageBox.Show(" Validar campos obligatorios");
        return false;
    }

    return true;
}
```

Ahora nos dirigimos al Form1.cs en donde creamos una instancia de LibroRepository para manejar los datos. El método CargarTabla carga todos los

```
3 referencias
public partial class Form1 : Form
{
    LibroRepository repo = new LibroRepository();
    3 referencias
    private void CargarTabla()
    {
        dataGridView1.DataSource = null;
        dataGridView1.DataSource = repo.ListarTodos();
    }
    1 referencia
    public Form1()
    {

        InitializeComponent();
        CargarTabla();
    }
```

libros en el DataGridView al iniciar el formulario.

El método LimpiarCampos se encarga de limpiar los campos del formulario. Borra los TextBox y reinicia el estado del CheckBox.

```
2 referencias
private void Limpiar()
{
    txtTitulo.Clear();
    txtAutor.Clear();
    txtAño.Clear();
    txtBuscarAutor.Clear();
    txtId.Clear();
    chkDisponible.Checked = false;
}
```

El botón Guardar valida los campos y crea un objeto Libro con los datos ingresados. Luego lo guarda en la base de datos, muestra un mensaje, actualiza la tabla y limpia los campos.

```
private void btnGuardar_Click(object sender, EventArgs e)
{
    if (!ValidarCampos()) return;

    Libro libro = new Libro
    {
        Titulo = txtTitulo.Text,
        Autor = txtAutor.Text,
        AñoPublicacion = int.Parse(txtAño.Text),
        Disponible = chkDisponible.Checked
    };
    repo.Guardar(libro);
    MessageBox.Show(" Guardado");
    CargarTabla();
    Limpiar();
}
```

El botón disponibles lbtiene la lista de libros disponibles. Si no hay resultados muestra un mensaje, y si hay, los muestra en el DataGridView.

```
private void btnDisponibles_Click(object sender, EventArgs e)
{
    var lista = repo.Disponibles();

    if (lista.Count == 0)
    {
        MessageBox.Show(" No hay libros disponibles");
    }
    dataGridView1.DataSource = lista;
}
```

El botón Buscar permite buscar libros por autor. Verifica que el campo no esté vacío y luego muestra los resultados en la tabla usando el método BuscarAutor.

```
1 referencia
private void btnBuscar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(txtBuscarAutor.Text))
    {
        MessageBox.Show(" Campo a bbuscar vacío");
        return;
    }

    dataGridView1.DataSource = repo.BuscarAutor(txtBuscarAutor.Text);
}
```

El boton actualizar verifica que el Id sea válido y actualiza la disponibilidad del libro. Luego muestra un mensaje y recarga los datos en la tabla.

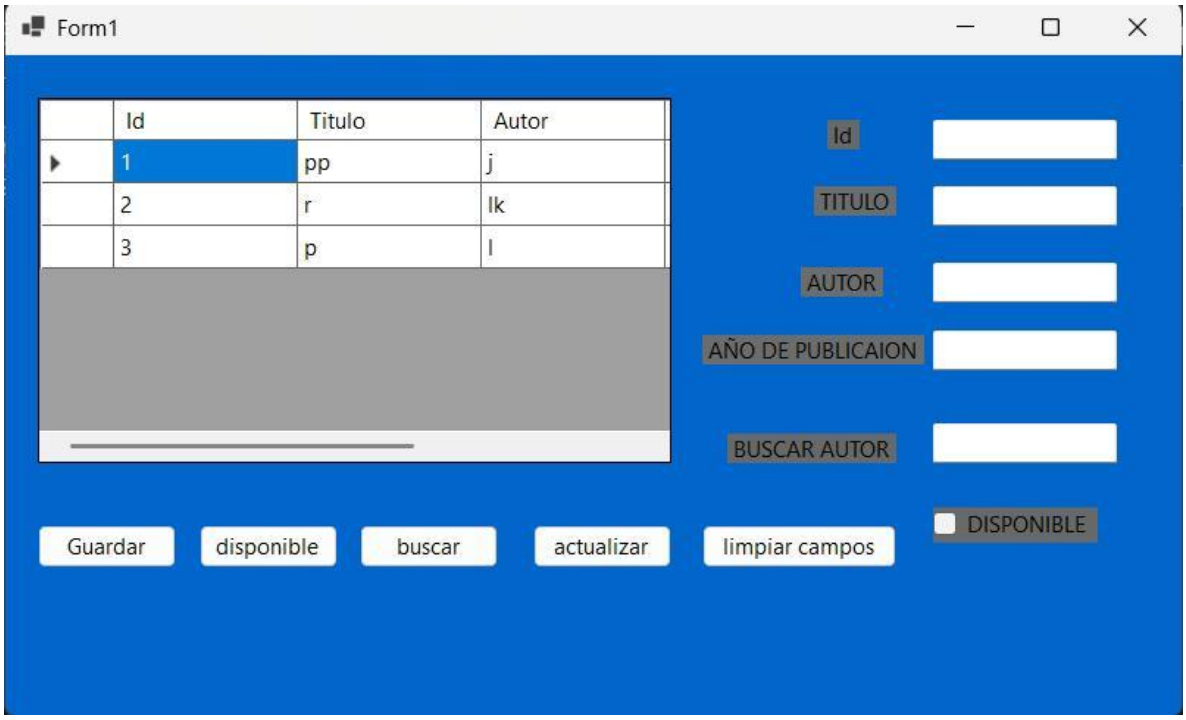
```
1 referencia
private void btnActualizar_Click(object sender, EventArgs e)
{
    int id;

    if (!int.TryParse(txtId.Text, out id))
    {
        MessageBox.Show("ID inválido");
        return;
    }

    repo.ActualizarDisponibilidad(id, chkDisponible.Checked);
    MessageBox.Show(" Actualizado");
    CargarTabla();
}
```

El botón limpiar llama al método Limpiar para borrar los campos del formulario.

Y el evento chkDisponible_CheckedChanged muestra automáticamente los libros disponibles en la tabla cuando se marca o desmarca el CheckBox.

```
1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    Limpiar();
}

1 referencia
private void chkDisponible_CheckedChanged(object sender, EventArgs e)
{
    dataGridView1.DataSource = repo.Disponibles();

}
```

Así se vería la ejecución de nuestro código



EJERCICIO 7

Esta es la clase Venta.cs, que en el desarrollo de software se conoce como una Entidad. Siguiendo el orden lógico que establecimos, esta es la primera parte que debes explicar en tu trabajo porque define qué es lo que el sistema va a procesar.

Bloque de Importaciones y Espacio de Nombres

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

- using System;: Importa las funciones básicas del sistema de .NET, como los tipos de datos fundamentales.
- using System.Collections.Generic;: Permite el uso de listas y colecciones de objetos.
- using System.Linq;: Proporciona herramientas para realizar consultas y filtrar datos de forma eficiente.
- using System.Text;: Contiene clases para el manejo y codificación de cadenas de texto.
- using System.Threading.Tasks;: Permite que el programa realice tareas en segundo plano (asíncronas) sin congelar la pantalla.

Espacio de Nombres y Definición

```
namespace SistemaDEventa7.Entidades
{
    7 referencias
    public  class Venta
    {
```

- namespace SistemaDEventa7.Entidades: Define la ubicación lógica del archivo dentro del proyecto, agrupándolo con otros modelos de datos.
- public class Venta: Declara una clase pública llamada "Venta", lo que permite que sea utilizada por el Repositorio para guardar datos y por el Formulario para capturarlos.

Propiedades de Datos

Estas líneas representan las columnas que normalmente encontrarías en tu tabla de base de datos:

```
1 referencia
public int Id { get; set; }
3 referencias
public string Cliente { get; set; }
3 referencias
public string Producto { get; set; }
4 referencias
public int Cantidad { get; set; }
4 referencias
public decimal Precio { get; set; }

0 referencias
public decimal Total => Cantidad * Precio;
```

- public int Id { get; set; }: Es el identificador único de la venta, esencial para poder buscar o eliminar registros específicos más adelante.
- public string Cliente y Producto: Son cadenas de texto que almacenan el nombre del comprador y el artículo vendido.
- public int Cantidad: Almacena el número entero de unidades vendidas.
- public decimal Precio: Utiliza el tipo decimal porque es el formato más preciso para manejar valores monetarios en C#.

Propiedad Calculada (Lógica de Negocio)

```
0 referencias
public decimal Total => Cantidad * Precio;
```

- public decimal Total => Cantidad * Precio;: Esta es una de las partes más importantes de la clase. No es una variable que el usuario escribe, sino que se calcula automáticamente multiplicando la cantidad por el precio cada vez que se solicita. Esto asegura que el total siempre sea correcto y esté actualizado en la interfaz.

La clase Venta actúa como el modelo de información del sistema. Su función es servir de 'molde' para que los datos viajen desde los cuadros de texto del formulario hacia la base de datos de manera organizada. Incluye una propiedad automática para el cálculo del total, lo que simplifica la lógica de la aplicación.

Esta es la clase Conexion.cs, que actúa como el puente de comunicación entre tu aplicación y el servidor de base de datos SQL Server. Siguiendo el orden, esta es la segunda parte (el canal de comunicación)

Bloque de Importaciones y Espacio de Nombres

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.Data.SqlClient;

namespace SistemaDEventa7.sqlConexion
{
```

- using System; hasta using System.Threading.Tasks;: Son librerías base de .NET que permiten el manejo de listas, textos y tareas del sistema.
- using Microsoft.Data.SqlClient;: Es la librería fundamental en este archivo, ya que proporciona las herramientas necesarias para conectarse y enviar comandos a bases de datos SQL Server.
- namespace SistemaDEventa7.sqlConexion: Define la ubicación lógica de la clase, agrupándola en una carpeta o espacio llamado "sqlConexion" para mantener el proyecto organizado.

Definición de la Clase

```
2 referencias
public class Conexion
{
```

- public class Conexion: Declara la clase como pública, lo que permite que sea instanciada por el repositorio (VentaRepository) para obtener acceso a la base de datos.

Configuración de la Conexión (Cadena de Conexión)

```
private string cadena = "Server=localhost\\SQLEXPRESS01;Database=SistemaVentas;Trusted_Connection=True;Encrypt=True;TrustServerCertificate=True;";

5 referencias
public SqlConnection ObtenerConexion()
{
    return new SqlConnection(cadena);
}
```

- private string cadena = "...": Define una variable privada de tipo texto que contiene la configuración técnica para encontrar el servidor.
    - Server=localhost\\SQLEXPRESS01;: Especifica que el servidor de base de datos está en la misma computadora (localhost) y usa la instancia llamada SQLEXPRESS01.
    - Database=SistemaVentas;: Indica el nombre de la base de datos a la cual se quiere ingresar.
    - Trusted_Connection=True;: Configura la conexión para usar la autenticación de Windows (no requiere usuario y contraseña manual).
    - Encrypt=True;: Activa el cifrado de los datos que viajan entre el programa y el servidor para mayor seguridad.
    - TrustServerCertificate=True;: Indica al programa que confíe en el certificado de seguridad del servidor local, evitando errores de validación durante el desarrollo.

Método de Conexión

```
5 referencias
public SqlConnection ObtenerConexion()
{
    return new SqlConnection(cadena);
}
```

- public SqlConnection ObtenerConexion(): Este método público devuelve un objeto de tipo SqlConnection. Este objeto es la "llave" física que abre la comunicación.
- return new SqlConnection(cadena);: Crea una nueva instancia del conector SQL utilizando la configuración definida en la variable cadena y se la entrega a quien la solicite (normalmente el repositorio).

La clase Conexion es la encargada de gestionar los parámetros técnicos necesarios para enlazar la aplicación con SQL Server. Al centralizar la cadena de conexión en este archivo, se facilita el mantenimiento del software, permitiendo cambiar de servidor o base de datos modificando una sola línea de código.

Esta es la clase VentaRepository.cs, que actúa como el "cerebro" de la aplicación para el manejo de datos. Aquí es donde se traduce la lógica de C# a instrucciones que SQL Server puede entender.

Encabezado y Configuración

```
using Microsoft.Data.SqlClient;
using SistemaDEventa7.Entidades;
using SistemaDEventa7.sqlConexion;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SistemaDEventa7.Repository
{
    2 referencias
    public  class VentaRepository
    {


        Conexion con = new Conexion();
```

- using Microsoft.Data.SqlClient;: Importa las herramientas necesarias para ejecutar comandos en SQL Server.
- using SistemaDEventa7.Entidades;: Permite que esta clase reconozca el objeto Venta.
- using SistemaDEventa7.sqlConexion;: Permite usar la clase que creamos para conectarnos a la base de datos.
- Conexion con = new Conexion();: Crea una instancia de nuestra clase de conexión para poder abrir el "puente" hacia la base de datos.

Método Guardar (Insertar registros)

```
1 referencia
public void Guardar(Venta v)
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        string query = "INSERT INTO Ventas (Cliente, Producto, Cantidad, Precio) VALUES (@Cliente,@Producto,@Cantidad,@Precio)";

        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@Cliente", v.Cliente);
        cmd.Parameters.AddWithValue("@Producto", v.Producto);
        cmd.Parameters.AddWithValue("@Cantidad", v.Cantidad);
        cmd.Parameters.AddWithValue("@Precio", v.Precio);

        cmd.ExecuteNonQuery();
    }
}
```

- public void Guardar(Venta v): Define el método que recibe un objeto venta para guardarlo.
- using (var conn = con.ObtenerConexion()): Abre una conexión segura que se cerrará automáticamente al terminar.
- conn.Open();: Activa físicamente la comunicación con el servidor.
- string query = "INSERT INTO Ventas...": Escribe la instrucción SQL. Los nombres con @ (como @Cliente) son parámetros de seguridad para evitar hackeos por inyección SQL.
- cmd.Parameters.AddWithValue(...): Reemplaza los parámetros @ por los datos reales que el usuario escribió en el formulario.
- cmd.ExecuteNonQuery();: Ejecuta la orden de insertar en la base de datos.

Método Listar (Consultar registros)

```
1 referencia
public List<Venta> Listar()
{
    List<Venta> lista = new List<Venta>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM Ventas", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Venta
            {
                Id = (int)dr["Id"],
                Cliente = dr["Cliente"].ToString(),
                Producto = dr["Producto"].ToString(),
                Cantidad = (int)dr["Cantidad"],
                Precio = (decimal)dr["Precio"]
            });
        }
    }
    return lista;
}
```

- public List<Venta> Listar(): Método que devuelve una lista de todas las ventas guardadas.
- SqlCommand cmd = new SqlCommand("SELECT * FROM Ventas", conn);: Crea la orden para pedir todos los datos de la tabla.
- SqlDataReader dr = cmd.ExecuteReader();: Crea un lector que recorre los resultados de SQL fila por fila.
- while (dr.Read()): Un ciclo que se repite mientras haya filas por leer.
- lista.Add(new Venta { ... });: Convierte los datos de SQL (como "Cliente" o "Precio") de nuevo en un objeto de C# y lo agrega a la lista.

Método Eliminar

```
1 referencia
public void Eliminar(int id)
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("DELETE FROM Ventas WHERE Id=@Id", conn);
        cmd.Parameters.AddWithValue("@Id", id);
        cmd.ExecuteNonQuery();
    }
}
```

- public void Eliminar(int id): Recibe el ID único de una venta para borrarla.
- DELETE FROM Ventas WHERE Id=@Id: Orden SQL que busca específicamente ese identificador y lo remueve de la tabla.

Métodos de Análisis (Cálculos en SQL)

```
1 referencia
public decimal TotalGeneral()
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT SUM(Cantidad * Precio) FROM Ventas", conn);

        object result = cmd.ExecuteScalar();
        return result == DBNull.Value ? 0 : Convert.ToDecimal(result);
    }
}
```

- TotalGeneral():
    - SELECT SUM(Cantidad * Precio): Pide a la base de datos que sume el valor total de todas las ventas.
    - ExecuteScalar(): Se usa porque solo esperamos un único número como resultado, no una lista.
    - result == DBNull.Value ? 0: Si no hay ventas, devuelve 0 para evitar errores.
- ProductoMasVendido():

- TOP 1 / GROUP BY / ORDER BY SUM(Cantidad) DESC: Esta consulta agrupa las ventas por producto, suma las cantidades de cada uno, las ordena de mayor a menor y nos da el nombre del primero de la lista.

La clase VentaRepository centraliza la lógica de acceso a datos utilizando el patrón Repository. Su función es ejecutar las operaciones CRUD (Crear, Leer, Eliminar) y realizar cálculos estadísticos directamente en el motor de base de datos, garantizando la integridad de la información mediante el uso de consultas parametrizadas.

Form.cs representa la Capa de Interfaz de Usuario. Aquí es donde se capturan las acciones del usuario y se muestran los resultados visualmente.

Declaraciones e Inicialización

```
using SistemaDEventa7.Entidades;
using SistemaDEventa7.Repository;

namespace SistemaDEventa7
{
    3 referencias
    public partial class Form1 : Form
    {

        VentaRepository repo = new VentaRepository();
```

- using SistemaDEventa7.Entidades;: Importa la clase Venta para que el formulario pueda crear objetos con ese formato.
- using SistemaDEventa7.Repository;: Permite al formulario comunicarse con el repositorio para gestionar los datos.

- public partial class Form1 : Form: Define la clase del formulario principal, la cual hereda las propiedades de una ventana de Windows (Form).
- VentaRepository repo = new VentaRepository();: Crea una instancia global del repositorio para que todos los botones puedan usar sus funciones de base de datos.

Métodos de Control de Datos

```
3 referencias
private void CargarDatos()
{
    dgvVentas.DataSource = null;
    dgvVentas.DataSource = repo.Listar();
}



1 referencia
public Form1()
{
    InitializeComponent();
    CargarDatos();
}
```

- private void CargarDatos(): Método interno encargado de refrescar la cuadrícula visual de ventas.
- dgvVentas.DataSource = null;: Limpia la fuente de datos actual para evitar conflictos visuales.

- dgvVentas.DataSource = repo.Listar();: Asigna la lista actualizada de ventas obtenida desde la base de datos al control visual DataGridView.
- public Form1(): Es el constructor; ejecuta InitializeComponent() para crear la interfaz y luego CargarDatos() para mostrar la información al abrir el programa.

Evento Guardar (btnGuardar_Click)

```
1 referencia
private void btnGuardar_Click(object sender, EventArgs e)
{
    Venta v = new Venta
    {
        Cliente = txtCliente.Text,
        Producto = txtProducto.Text,
        Cantidad = int.Parse(txtCantidad.Text),
        Precio = decimal.Parse(txtPrecio.Text)
    };

    repo.Guardar(v);
    CargarDatos();



}
```

- Venta v = new Venta { ... };: Crea un nuevo objeto de tipo Venta en memoria.
- Cliente = txtCliente.Text: Asigna el texto escrito en el cuadro de texto del cliente al objeto.
- int.Parse(txtCantidad.Text): Convierte el texto de la cantidad en un número entero.

- decimal.Parse(txtPrecio.Text): Convierte el texto del precio en un número decimal.
- repo.Guardar(v);: Envía el objeto lleno al repositorio para insertarlo en la base de datos.
- CargarDatos();: Actualiza la tabla para mostrar la nueva venta inmediatamente.

Evento Calcular Total (btnTotalVenta_Click)

```
1 referencia
private void btnTotalVenta_Click(object sender, EventArgs e)
{


    if (string.IsNullOrWhiteSpace(txtCantidad.Text) || string.IsNullOrWhiteSpace(txtPrecio.Text))
    {
        MessageBox.Show("Ingrese cantidad y precio");
        return;
    }

    if (!int.TryParse(txtCantidad.Text, out int cantidad))
    {
        MessageBox.Show("Cantidad inválida");
        return;
    }

    if (!decimal.TryParse(txtPrecio.Text, out decimal precio))
    {
        MessageBox.Show("Precio inválido");
        return;
    }

    decimal total = cantidad * precio;

    MessageBox.Show("Total venta: " + total);
}
```

- if (string.IsNullOrWhiteSpace(...)): Verifica que los campos necesarios no estén vacíos para evitar errores.
- int.TryParse(...) y decimal.TryParse(...): Intenta convertir el texto a número de forma segura; si el usuario escribió letras, muestra un mensaje de error.

- decimal total = cantidad * precio;: Realiza el cálculo matemático del total de la venta actual.
- MessageBox.Show(...): Muestra una pequeña ventana emergente con el resultado del cálculo.

Otros Eventos (Reportes y Limpieza)

```
1 referencia
private void btnTotalGeneral_Click(object sender, EventArgs e)
{

    MessageBox.Show("Total general: " + repo.TotalGeneral());
}

1 referencia
private void btnProductoMasVendido_Click(object sender, EventArgs e)
{
    MessageBox.Show("Más vendido: " + repo.ProductoMasVendido());
}

1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtCliente.Clear();
    txtProducto.Clear();
    txtCantidad.Clear();
    txtPrecio.Clear();
}

1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    int id = int.Parse(txtId.Text);
    repo.Eliminar(id);
    CargarDatos();



}
```

- btnTotalGeneral_Click: Muestra el acumulado total de todas las ventas consultando directamente al repositorio.
- btnProductoMasVendido_Click: Muestra el nombre del producto que tiene más unidades vendidas según el análisis del repositorio.
- btnLimpiar_Click: Utiliza el método .Clear() para vaciar todos los cuadros de texto de la pantalla.
- btnEliminar_Click: Toma el ID escrito por el usuario, le ordena al repositorio borrar ese registro y refresca la tabla visual.

El archivo Form1.cs actúa como el mediador entre el usuario y la lógica de datos. Su función principal es validar las entradas del usuario, convertir los datos de texto a formatos numéricos y llamar a los métodos del repositorio para realizar cambios permanentes en la base de datos, manteniendo la interfaz siempre actualizada.

BASE DE DATOS

SQLQuery1.s...subot (84))*

```sql
CREATE DATABASE SistemaVentas;
GO

USE SistemaVentas;
GO

CREATE TABLE Ventas (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Cliente NVARCHAR(100) NOT NULL,
    Producto NVARCHAR(100) NOT NULL,
    Cantidad INT NOT NULL CHECK (Cantidad > 0),
    Precio DECIMAL(10,2) NOT NULL CHECK (Precio > 0)
);

```

Este es el Script SQL para tu primer proyecto, el de ventas. Es el código que "prepara el terreno" en SQL Server para que la aplicación pueda guardar la información de cada transacción.

Aquí tienes la explicación detallada:

Creación de la Base de Datos

- CREATE DATABASE SistemaVentas;: Esta instrucción crea el archivo principal en el servidor donde se almacenará todo lo relacionado con las ventas.

- GO: Le indica a SQL Server que termine de crear la base de datos antes de pasar a la siguiente instrucción.

Selección de la Base de Datos

- USE SistemaVentas;: Le dice al sistema: "De todas las bases de datos que existen, quiero trabajar específicamente en SistemaVentas".

Estructura de la Tabla Ventas

Aquí defines las columnas que guardarán los datos. Nota que coinciden con las propiedades de tu clase Venta.cs en C#.

- Id INT IDENTITY(1,1) PRIMARY KEY:
  - INT: Es un número entero.
  - IDENTITY(1,1): Autoincremento. El primer registro será el 1, el segundo el 2, y así sucesivamente. Pepe, tú no necesitas enviarle el ID desde C#; SQL lo genera solo.
  - PRIMARY KEY: Es el documento de identidad de la fila; no puede haber dos ventas con el mismo ID.
- Cliente y Producto (NVARCHAR(100)): Son campos de texto que aceptan hasta 100 caracteres. El NOT NULL significa que no puedes dejar estos campos vacíos al guardar.
- Cantidad INT NOT NULL CHECK (Cantidad > 0):
  - CHECK (Cantidad > 0): Esta es una regla de seguridad. Si el programa intenta guardar una cantidad de 0 o negativa, SQL bloqueará la operación para evitar errores lógicos.
- Precio DECIMAL(10,2) NOT NULL CHECK (Precio > 0):
  - DECIMAL(10,2): Significa que el precio puede tener hasta 10 dígitos en total, de los cuales 2 son decimales (ejemplo: 99,999,999.99). También tiene una regla CHECK para que el precio siempre sea mayor a cero.

El script de base de datos para el sistema de ventas establece una estructura de almacenamiento robusta mediante el uso de restricciones (CONSTRAINTS). La implementación de campos IDENTITY automatiza la indexación, mientras que las cláusulas CHECK garantizan que no se ingresen datos erróneos, como cantidades o precios negativos, directamente desde el nivel del servidor.

EJERCICIO 8

Este código corresponde a la clase Paciente.cs, que es la Entidad o modelo de datos de este nuevo sistema médico. Al igual que en el proyecto anterior, su función es definir la estructura de la información que se va a manejar.

Espacio de Nombres y Definición

```
namespace SistemaMedico8.Entidades
{
    15 referencias
    public class Paciente
    {
```

- namespace SistemaMedico8.Entidades: Define la ubicación lógica del archivo. En este caso, indica que pertenece al proyecto "SistemaMedico8" y está agrupado en la carpeta de "Entidades".
- public class Paciente: Declara una clase pública llamada Paciente. Al ser pública, otros componentes del sistema (como el formulario o la base de datos) pueden crear objetos basados en este molde.

Propiedades (Atributos del Paciente)

Estas líneas definen qué datos se guardarán de cada persona. El uso de { get; set; } permite que los valores sean leídos y modificados por el programa.

```
public int Id { get; set; }
5 referencias
public string Nombre { get; set; }
5 referencias
public int Edad { get; set; }
5 referencias
public string Diagnostico { get; set; }
```

- public int Id { get; set; }: Es una propiedad de tipo entero que sirve como el identificador único del paciente. Es fundamental para buscar, editar o eliminar a una persona específica en la base de datos sin errores.
- public string Nombre { get; set; }: Define una cadena de texto para almacenar el nombre completo del paciente.
- public int Edad { get; set; }: Almacena un número entero que representa los años del paciente.
- public string Diagnostico { get; set; }: Es una cadena de texto destinada a guardar la descripción médica o la enfermedad detectada en el paciente.

La clase Paciente representa la entidad principal del sistema médico. Define los atributos esenciales que se almacenarán en la base de datos: un identificador único (Id), el nombre, la edad y el diagnóstico médico. Esta estructura permite que la información viaje de manera organizada entre la interfaz de usuario y la capa de persistencia de datos.

Importaciones y Espacio de Nombres

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.Data.SqlClient;

namespace SistemaMedico8.sqlConexion
{
```

- using System; (y las siguientes 4 líneas): Son librerías base de .NET que permiten al programa manejar textos, listas y tareas generales.
- using Microsoft.Data.SqlClient;: Es la librería esencial que contiene las clases necesarias para interactuar con SQL Server, como el objeto de conexión.
- namespace SistemaMedico8.sqlConexion: Organiza esta clase dentro de un espacio de nombres específico para las conexiones del sistema médico, evitando que se mezcle con otras partes del código.

Esta es la clase Conexion.cs para el nuevo proyecto SistemaMedico8. Su función es establecer el canal de comunicación técnica entre tu aplicación y el servidor de base de datos SQL Server.

Estructura de la Clase

```
2 referencias
public class Conexion
{
```

- public class Conexion: Declara la clase como pública para que pueda ser utilizada por el repositorio de pacientes cuando este necesite guardar o leer datos.

La Cadena de Conexión

```
private string cadena = "Server=localhost\\SQLEXPRESS01;Database=SistemaMedico;Trusted_Connection=True;Encrypt=True;TrustServerCertificate=True;";

4 referencias
public SqlConnection ObtenerConexion()
{
    return new SqlConnection(cadena);
}
```

- private string cadena = "...": Esta variable contiene la configuración de acceso.
    - Server=localhost\\SQLEXPRESS01;: Indica que la base de datos está en tu computadora local, en la instancia de SQL llamada SQLEXPRESS01.
    - Database=SistemaMedico;: Especifica el nombre de la base de datos a la que nos vamos a conectar (en este caso, una distinta a la anterior).

    - Trusted_Connection=True;: Indica que se usará la autenticación de Windows del usuario actual.
    - Encrypt=True; y TrustServerCertificate=True;: Configuran la seguridad del canal de datos para que la información viaje cifrada y se confíe en el certificado del servidor local.

- public SqlConnection ObtenerConexion(): Define un método público que devuelve un objeto SqlConnection. Este objeto representa la conexión física hacia el servidor.
- return new SqlConnection(cadena);: Crea y entrega una nueva instancia del conector SQL usando la configuración de la variable cadena.

Resumen para tu Word:

La clase Conexion actúa como el gestor de acceso al motor de base de datos SQL Server para el proyecto SistemaMedico8. Define la ruta del servidor y el nombre de la base de datos, proporcionando un método estandarizado para que el sistema abra canales de comunicación cada vez que se requiera realizar una operación médica.

Esta es la clase PacienteRepository.cs. Al igual que en el proyecto anterior, este es el "cerebro" que maneja los datos, pero ahora adaptado a un entorno médico. Aquí se definen las reglas para interactuar con la tabla de pacientes en SQL Server.

Configuración Inicial

```
using Microsoft.Data.SqlClient;
using SistemaMedico8.Entidades;
using SistemaMedico8.sqlConexion;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SistemaMedico8.Repository
{
    2 referencias
```

- using Microsoft.Data.SqlClient;: Carga las herramientas para ejecutar comandos SQL.
- Conexion con = new Conexion();: Crea el objeto que nos permite abrir el "puente" hacia la base de datos SistemaMedico.

Método Guardar (Insertar Paciente)

```
1 referencia
public void Guardar(Paciente p)
{
    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        string query = "INSERT INTO Pacientes (Nombre, Edad, Diagnostico) VALUES (@Nombre,@Edad,@Diagnostico)";
        SqlCommand cmd = new SqlCommand(query, conn);

        cmd.Parameters.AddWithValue("@Nombre", p.Nombre);
        cmd.Parameters.AddWithValue("@Edad", p.Edad);
        cmd.Parameters.AddWithValue("@Diagnostico", p.Diagnostico);

        cmd.ExecuteNonQuery();
    }
}
```

- public void Guardar(Paciente p): Recibe un objeto Paciente con sus datos llenos.
- using (var conn = con.ObtenerConexion()): Garantiza que la conexión se cierre sola al terminar para no desperdiciar memoria.
- INSERT INTO Pacientes (...): Es la instrucción SQL para crear un nuevo registro.
- cmd.Parameters.AddWithValue(...): Vincula los datos del objeto (Nombre, Edad, Diagnóstico) con la consulta de forma segura, evitando ataques informáticos.

Método Listar (Ver todos)

```
public List<Paciente> Listar()
{
    List<Paciente> lista = new List<Paciente>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM Pacientes", conn);
        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Paciente
            {
                Id = (int)dr["Id"],
                Nombre = dr["Nombre"].ToString(),
                Edad = (int)dr["Edad"],
                Diagnostico = dr["Diagnostico"].ToString()
            });
        }
    }
    return lista;
}
```

- public List<Paciente> Listar(): Devuelve una lista con todos los pacientes registrados.
- SqlDataReader dr = cmd.ExecuteReader();: Crea un lector que recorre la tabla fila por fila.

- lista.Add(new Paciente { ... });: Por cada fila encontrada en SQL, crea un objeto Paciente en C# y lo guarda en la lista.

Método BuscarPorNombre (Filtro dinámico)

```
1 referencia
public List<Paciente> BuscarPorNombre(string nombre)
{
    List<Paciente> lista = new List<Paciente>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        string query = "SELECT * FROM Pacientes WHERE Nombre LIKE @Nombre";
        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@Nombre", "%" + nombre + "%");

        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Paciente
            {
                Id = (int)dr["Id"],
                Nombre = dr["Nombre"].ToString(),
                Edad = (int)dr["Edad"],
                Diagnostico = dr["Diagnostico"].ToString()
            });
        }
    }
    return lista;
}
```

- public List<Paciente> BuscarPorNombre(string nombre): Permite buscar pacientes escribiendo solo una parte de su nombre.
- WHERE Nombre LIKE @Nombre: Usa el operador LIKE de SQL.
- "%" + nombre + "%": Los porcentajes sirven como comodines. Si buscas "Ana", encontrará a "Ana Maria", "Mariana" o "Ana".

Método MayoresDe60 (Reporte Específico)

```
public List<Paciente> MayoresDe60()
{
    List<Paciente> lista = new List<Paciente>();

    using (var conn = con.ObtenerConexion())
    {
        conn.Open();

        string query = "SELECT * FROM Pacientes WHERE Edad > 60";
        SqlCommand cmd = new SqlCommand(query, conn);

        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Paciente
            {
                Id = (int)dr["Id"],
                Nombre = dr["Nombre"].ToString(),
                Edad = (int)dr["Edad"],
                Diagnostico = dr["Diagnostico"].ToString()
            });
        }
    }
    return lista;
}
```

- public List<Paciente> MayoresDe60(): Este es un método de consulta filtrada.
- WHERE Edad > 60: La lógica se traslada directamente a SQL para que el servidor solo nos entregue los registros de personas de la tercera edad. Es muy útil para reportes médicos rápidos.

La clase PacienteRepository gestiona la persistencia de datos del sistema médico. Además de las funciones básicas de guardado y lectura, incorpora métodos de búsqueda avanzada y filtrado por criterios específicos (como la

edad), optimizando la recuperación de información directamente desde el motor de base de datos mediante lenguaje Transact-SQL.

Form.cs. Este archivo representa la Capa de Presentación, donde los datos se convierten en elementos visuales para el usuario.

Inicialización y Configuración

```
PacienteRepository repo = new PacienteRepository();

2 referencias
private void CargarDatos()
{
    dgvPacientes.DataSource = null;
    dgvPacientes.DataSource = repo.Listar();
}
```

- PacienteRepository repo = new PacienteRepository();: Instancia el repositorio para que el formulario pueda enviar y recibir datos de la base de datos.
- private void CargarDatos(): Este método limpia la fuente de datos del DataGridView y le asigna la lista completa de pacientes obtenida del repositorio, refrescando la tabla visual.
- public Form1(): El constructor de la ventana. Ejecuta la configuración visual inicial (InitializeComponent) y llena la tabla de pacientes automáticamente al abrir el programa.

Evento Guardar (btnGuardar_Click)

```
1 referencia
private void btnGuardar_Click(object sender, EventArgs e)
{
    if (!int.TryParse(txtEdad.Text, out int edad))
    {
        MessageBox.Show("Edad inválida");
        return;
    }

    Paciente p = new Paciente
    {
        Nombre = txtNombre.Text,
        Edad = edad,
        Diagnostico = txtDiagnostico.Text
    };

    repo.Guardar(p);
    CargarDatos();
}
```

- if (!int.TryParse(txtEdad.Text, out int edad)): Realiza una validación crítica. Intenta convertir el texto del cuadro de edad en un número; si el usuario ingresa algo que no es un número, detiene el proceso y muestra un mensaje de error.
- Paciente p = new Paciente { ... };: Crea un nuevo objeto de la entidad Paciente y le asigna los valores capturados en los cuadros de texto (txtNombre, txtEdad, txtDiagnostico).
- repo.Guardar(p);: Llama al método del repositorio para insertar permanentemente el nuevo paciente en SQL Server.

- CargarDatos();: Actualiza la cuadrícula en pantalla para incluir al paciente recién registrado.

Eventos de Búsqueda y Filtros

```
private void btnBuscar_Click(object sender, EventArgs e)
{
    dgvPacientes.DataSource = repo.BuscarPorNombre(txtBuscar.Text);
}

1 referencia
private void btnMayores60_Click(object sender, EventArgs e)
{
    dgvPacientes.DataSource = repo.MayoresDe60();
}
```

- btnBuscar_Click: Toma el texto ingresado en txtBuscar y actualiza la tabla mostrando solo los resultados que coincidan con ese nombre, utilizando el método de búsqueda del repositorio.
- btnMayores60_Click: Ejecuta el filtro especializado del repositorio para mostrar en pantalla únicamente a los pacientes que superan los 60 años.

Mantenimiento de Interfaz (btnLimpiar_Click)

```
1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtId.Clear();
    txtNombre.Clear();
    txtEdad.Clear();
    txtDiagnostico.Clear();
    txtBuscar.Clear();
}
```

- txtId.Clear(); (y líneas siguientes): Utiliza la función .Clear() para vaciar todos los campos de entrada de la interfaz, dejándolos listos para un nuevo registro o búsqueda.

El archivo Form1.cs gestiona la interacción directa con el usuario en el entorno médico. Su responsabilidad incluye la validación de tipos de datos (como asegurar que la edad sea un número), la ejecución de filtros de búsqueda dinámicos y la presentación organizada de la información clínica mediante controles de datos enlazados al repositorio.

SQLQuery2.s...subot (89))* | SQLQuery1.sql...Z\subot (84))

```sql
CREATE DATABASE SistemaMedico;
GO

USE SistemaMedico;
GO

CREATE TABLE Pacientes (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Nombre NVARCHAR(100) NOT NULL,
    Edad INT NOT NULL CHECK (Edad > 0),
    Diagnostico NVARCHAR(200) NOT NULL
);
```

Este código es el Script SQL que crea la base de datos y la tabla necesarias para que tu sistema de pacientes funcione. Es el fundamento de todo, ya que aquí es donde se define cómo se guardará físicamente la información en el disco duro.

Aquí tienes la explicación paso a paso:

Creación de la Base de Datos

- `CREATE DATABASE SistemaMedico;`: Esta instrucción le ordena a SQL Server crear un nuevo contenedor llamado "SistemaMedico" para organizar todas las tablas relacionadas con este proyecto.
- `GO`: Es un comando que le dice al servidor: "Ejecuta lo anterior ahora mismo antes de seguir con la siguiente instrucción".

Selección de Contexto

- `USE SistemaMedico;`: Indica que todas las acciones siguientes (como crear tablas o insertar datos) se deben realizar dentro de la base de datos que acabamos de crear.

- GO: Nuevamente, asegura que el cambio de base de datos se procese antes de intentar crear la tabla.

Creación de la Tabla Pacientes

Aquí es donde defines las columnas que coinciden con las propiedades que pusiste en tu clase Paciente.cs de C#.

- Id INT IDENTITY(1,1) PRIMARY KEY:
  - INT: Define que es un número entero.
  - IDENTITY(1,1): Esto es clave. Significa que SQL generará el ID automáticamente (empezando en 1 y sumando 1 cada vez). Pepe, tú no necesitas escribir el ID en el programa; SQL lo hace por ti.
  - PRIMARY KEY: Marca esta columna como el identificador único y principal de la tabla.
- Nombre NVARCHAR(100) NOT NULL:
  - NVARCHAR(100): Reserva espacio para texto de hasta 100 caracteres.
  - NOT NULL: Obliga a que siempre se escriba un nombre; no permite dejarlo vacío.
- Edad INT NOT NULL CHECK (Edad > 0):
  - CHECK (Edad > 0): Esta es una regla de validación. SQL rechazará cualquier intento de guardar a un paciente si la edad es 0 o negativa.
- Diagnostico NVARCHAR(200) NOT NULL: Reserva espacio para un texto más largo (200 caracteres) para la descripción médica y también es obligatorio.

El script SQL define la persistencia de datos mediante la creación de la base de datos SistemaMedico y la tabla Pacientes. Se utiliza una estructura de integridad de datos mediante una llave primaria autoincremental (IDENTITY) y restricciones de validación (CHECK) para asegurar que la información almacenada sea lógica y consistente.

EJERCICIO 9

CLASE DetalleFactura.cs

La clase DetalleFactura pertenece al sistema de facturación y se utiliza para almacenar la información de cada producto agregado a una factura. Cada objeto creado a partir de esta clase representa un detalle dentro de la factura, incluyendo el nombre del producto, la cantidad, el precio y el total.

El código inicia importando varias librerías del sistema:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

Estas librerías permiten utilizar herramientas y funciones básicas de C#, como manejo de texto, listas, consultas de datos y tareas del sistema.

Posteriormente se define el namespace:

```
namespace FacturarisacionContransicion9.Entidades
```

El namespace sirve para organizar las clases del proyecto. En este caso, la clase pertenece al espacio de nombres FacturarisacionContransicion9.Entidades, donde se almacenan las entidades o modelos de datos del sistema.

Luego se declara la clase:

```
public  class DetalleFactura
{

    4 referencias
    public string Producto { get; set; }
    5 referencias
    public int Cantidad { get; set; }
    5 referencias
    public decimal Precio { get; set; }

    1 referencia
    public decimal Total => Cantidad * Precio;



}
```

La palabra public indica que la clase puede utilizarse desde cualquier parte del proyecto. La clase DetalleFactura representa la información de un producto dentro de una factura.

Dentro de la clase se encuentra la propiedad Producto:

```
public string Producto { get; set; }
```

Esta propiedad almacena el nombre del producto. El tipo de dato string se utiliza para guardar texto. Las palabras get y set permiten obtener y asignar valores a la propiedad.

Después se encuentra la propiedad Cantidad:

```
public int Cantidad { get; set; }
```

Esta propiedad almacena la cantidad de productos facturados. El tipo int representa números enteros.

A continuación se declara la propiedad Precio:

```
public decimal Precio { get; set; }
```

Esta propiedad guarda el precio del producto. Se utiliza el tipo decimal porque es el más adecuado para trabajar con valores monetarios y decimales.

Finalmente se encuentra la propiedad Total:

```
public decimal Total => Cantidad * Precio;
```

Esta es una propiedad calculada que obtiene automáticamente el total multiplicando la cantidad por el precio del producto. De esta manera, el sistema calcula el total sin necesidad de ingresarlo manualmente.

Por ejemplo, si la cantidad es 2 y el precio es 15, el total será 30.

En conclusión, la clase DetalleFactura permite representar cada producto dentro de una factura, almacenando la información necesaria y calculando automáticamente el total correspondiente.

CLASE Factura.cs

La clase Factura pertenece al sistema de facturación y se utiliza para almacenar la información general de una factura. Esta clase guarda datos importantes como el número de factura, el nombre del cliente, la fecha y la lista de productos incluidos en la factura.

El código inicia importando varias librerías del sistema:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

Estas librerías permiten utilizar funciones básicas de C#, listas, manejo de texto y otras herramientas necesarias para el funcionamiento del programa.

Posteriormente se define el namespace:

```
namespace FacturarisacionContransicion9.Entidades
```

El namespace sirve para organizar las clases dentro del proyecto. En este caso, la clase pertenece al espacio de nombres FacturarisacionContransicion9.Entidades, donde se almacenan las entidades principales del sistema.

Luego se declara la clase:

```
public class Factura
{

    0 referencias
    public int Id { get; set; }
    4 referencias
    public string Cliente { get; set; }
    4 referencias
    public DateTime Fecha { get; set; }
    3 referencias
    public List<DetalleFactura> Detalles { get; set; } = new List<DetalleFactura>();

}
```

```
public class Factura
```
La palabra public indica que la clase puede ser utilizada desde cualquier parte del programa. La clase Factura representa la información principal de una factura.

Dentro de la clase se encuentra la propiedad Id:

```
public int Id { get; set; }
```
Esta propiedad almacena el identificador o número de la factura. El tipo int se utiliza porque el identificador es un número entero.

Después se encuentra la propiedad Cliente:

```
public string Cliente { get; set; }
```
Esta propiedad almacena el nombre del cliente al que pertenece la factura. El tipo string se utiliza para guardar texto.

A continuación se declara la propiedad Fecha:

```
public DateTime Fecha { get; set; }
```
Esta propiedad almacena la fecha en que se realizó la factura. El tipo DateTime permite trabajar con fechas y horas en C#.

Finalmente se encuentra la propiedad Detalles:

```
public List<DetalleFactura> Detalles { get; set; } = new List<DetalleFactura>();
```
Esta propiedad almacena una lista de objetos de tipo DetalleFactura.

La clase DetalleFactura contiene la información de cada producto agregado a la factura, como nombre del producto, cantidad, precio y total.

La palabra List indica que se puede almacenar más de un elemento dentro de la lista.

La instrucción:

```
new List<DetalleFactura>();
```

crea una lista vacía lista para agregar productos.

CLASE FacturaRepository.cs

La clase FacturaRepository se encarga de guardar las facturas y sus detalles en la base de datos SQL Server. Su función principal es conectar el programa con la base de datos y ejecutar las consultas necesarias para almacenar la información.

El código inicia importando librerías necesarias para trabajar con las entidades del sistema, la conexión SQL y SQL Server:

```
using FacturarisacionContransicion9.Entidades;
using FacturarisacionContransicion9.sqlConexion;
using Microsoft.Data.SqlClient;
```

Luego se define el namespace Repository, el cual contiene las clases encargadas de acceder a la base de datos.

```
namespace FacturarisacionContransicion9.Repository
{
```

Después se declara la clase:

```
public class FacturaRepository
{



    Conexion con = new Conexion();

    1 referencia
    public void GuardarFactura(Factura factura)
    {
        using (var conn = con.ObtenerConexion())
        {
            conn.Open();
            var trans = conn.BeginTransaction();

            try
            {

                var cmdFactura = new SqlCommand(
                    "INSERT INTO Facturas (Cliente,Fecha) OUTPUT INSERTED.Id VALUES (@c,@f)",
                    conn, trans);

                cmdFactura.Parameters.AddWithValue("@c", factura.Cliente);
                cmdFactura.Parameters.AddWithValue("@f", factura.Fecha);

                int facturaId = (int)cmdFactura.ExecuteScalar();


                foreach (var d in factura.Detalles)
                {
                    var cmdDetalle = new SqlCommand(
                        "INSERT INTO DetalleFactura (FacturaId,Producto,Cantidad,Precio) VALUES (@id,@p,@c,@pr)",
                        conn, trans);

                    cmdDetalle.Parameters.AddWithValue("@id", facturaId);
                    cmdDetalle.Parameters.AddWithValue("@p", d.Producto);
                    cmdDetalle.Parameters.AddWithValue("@c", d.Cantidad);
                    cmdDetalle.Parameters.AddWithValue("@pr", d.Precio);

                    cmdDetalle.ExecuteNonQuery();
                }

                trans.Commit();
            }
            catch
            {
                trans.Rollback();
                throw;
            }
        }
```

```
public class FacturaRepository
{
```

Dentro de la clase se crea un objeto de tipo Conexion:

```
Conexion con = new Conexion();
```

Este objeto se utiliza para obtener la conexión con SQL Server.

El método principal de la clase es:

```
public void GuardarFactura(Factura factura)
{
```

Este método recibe un objeto Factura y guarda toda su información en la base de datos.

Primero se abre la conexión y se inicia una transacción:

```
conn.Open();
var trans = conn.BeginTransaction();
```

La transacción garantiza que todos los datos se guarden correctamente. Si ocurre un error, los cambios se cancelan.

Luego se ejecuta un comando SQL para insertar la factura:

```
"INSERT INTO Facturas (Cliente,Fecha) OUTPUT INSERTED.Id VALUES (@c,@f)",
```

Después se obtiene el Id generado de la factura y se guarda en una variable.

Posteriormente se utiliza un ciclo foreach para recorrer todos los productos de la factura:

```
foreach (var d in factura.Detalles)
```

Por cada producto se ejecuta otro comando SQL que inserta:

```
cmdDetalle.Parameters.AddWithValue("@id", facturaId);
cmdDetalle.Parameters.AddWithValue("@p", d.Producto);
cmdDetalle.Parameters.AddWithValue("@c", d.Cantidad);
cmdDetalle.Parameters.AddWithValue("@pr", d.Precio);
```

Finalmente, si todo sale bien, se confirma la transacción:

```
trans.Commit();
```

Si ocurre algún error, se cancelan todos los cambios utilizando:

```
trans.Rollback();
```

En conclusión, la clase FacturaRepository permite guardar una factura completa junto con todos sus productos en la base de datos, utilizando transacciones para asegurar que la información se almacene correctamente.

CLASE Conexión.cs

La clase Conexion se utiliza para establecer la conexión entre el programa y la base de datos SQL Server. Su función principal es proporcionar una conexión lista para utilizarse en otras partes del sistema.

El código inicia importando librerías necesarias del sistema y la librería de SQL Server:

```
using Microsoft.Data.SqlClient;
```

Esta librería permite trabajar con bases de datos SQL Server desde C#.

Luego se define el namespace:

```
namespace FacturarisacionContransicion9.sqlConexion
```

El namespace sqlConexion se utiliza para almacenar las clases relacionadas con la conexión a la base de datos.

Después se declara la clase:

```
public class Conexion
```

Esta clase contiene la configuración necesaria para conectarse a SQL Server.

Dentro de la clase se declara una variable llamada cadena:

```
private string cadena = "Server=localhost\\SQLEXPRESS01;Database=SistemaFacturacion;Trusted_Connection=True;Encrypt=True;TrustServerCertificate=True;";
```

Esta variable almacena la cadena de conexión.

La cadena de conexión contiene:

- El nombre del servidor SQL (localhost\SQLEXPRESS01)
- El nombre de la base de datos (SistemaFacturacion)
- El tipo de autenticación utilizada

La palabra private significa que esta variable solo puede utilizarse dentro de la clase Conexion.

Luego se crea el método:

```
public SqlConnection ObtenerConexion()
{
    return new SqlConnection(cadena);
}
```

Este método devuelve una conexión de tipo SqlConnection.

Finalmente:

```
return new SqlConnection(cadena);
```

Esta línea crea y retorna una nueva conexión utilizando la cadena de conexión definida anteriormente.

En conclusión, la clase Conexion centraliza la configuración de acceso a SQL Server y permite que otras clases del sistema obtengan fácilmente una conexión a la base de datos.